

EJEMPLOS: DIAGRAMAS UML

1	<i>Clasificación de los diagramas UML</i>	2
2	<i>Diagramas de clases</i>	2
2.1	Modelo de dominio de una biblioteca	2
2.2	Modelo de dominio de un sistema de compras on-line	4
2.3	Modelo de dominio de la gestión de un hospital	5
2.4	Diagrama de clases que ilustra el uso de la API de cámara Android	8
3	<i>Diagrama de objetos</i>	9
3.1	Controlador de inicio de sesión en un sistema de compras on-line	9
4	<i>Diagrama de paquetes</i>	10
4.1	API de la plataforma Java 7 Standard Edition	10
4.2	API de Java Servlet 3.0	12
4.3	Aplicación multi-capa	14
4.4	Arquitectura web multi-capa	15
5	<i>Diagramas de casos de uso</i>	16
5.1	Área de embarque y control de seguridad	16
5.2	Cajero automático	17
5.3	Compras on-line	18
6	<i>Diagramas de actividad</i>	20
6.1	Máquina expendedora de billetes	20
6.2	Proceso de gestión de documentos	22
6.3	Compra on-line	22
7	<i>Diagramas de secuencia</i>	23
7.1	Compra on-line	23
7.2	Proporcionar comentarios en Pluck	24
7.3	Autorización de usuario Web por Facebook	25
8	<i>Diagramas de comunicación</i>	27
8.1	Compra on-line	27
9	<i>Diagramas de estados</i>	27
9.1	Estados del agua	27
9.2	Cajero automático	28
9.3	Cuenta de usuario en un sistema de compras on-line	29
10	<i>Bibliografía</i>	30

Cada elemento físico de la biblioteca (libro, CD, DVD, etc.) podría tener su propio número de elemento, que podría estar soportado en un código de barras. El propósito de los códigos de barras es proporcionar un identificador único y escaneable que asocie el elemento físico con el registro electrónico del catálogo. El código de barras debe estar unido físicamente al elemento, y el número de código de barras se introduce en el campo correspondiente del registro electrónico del artículo.

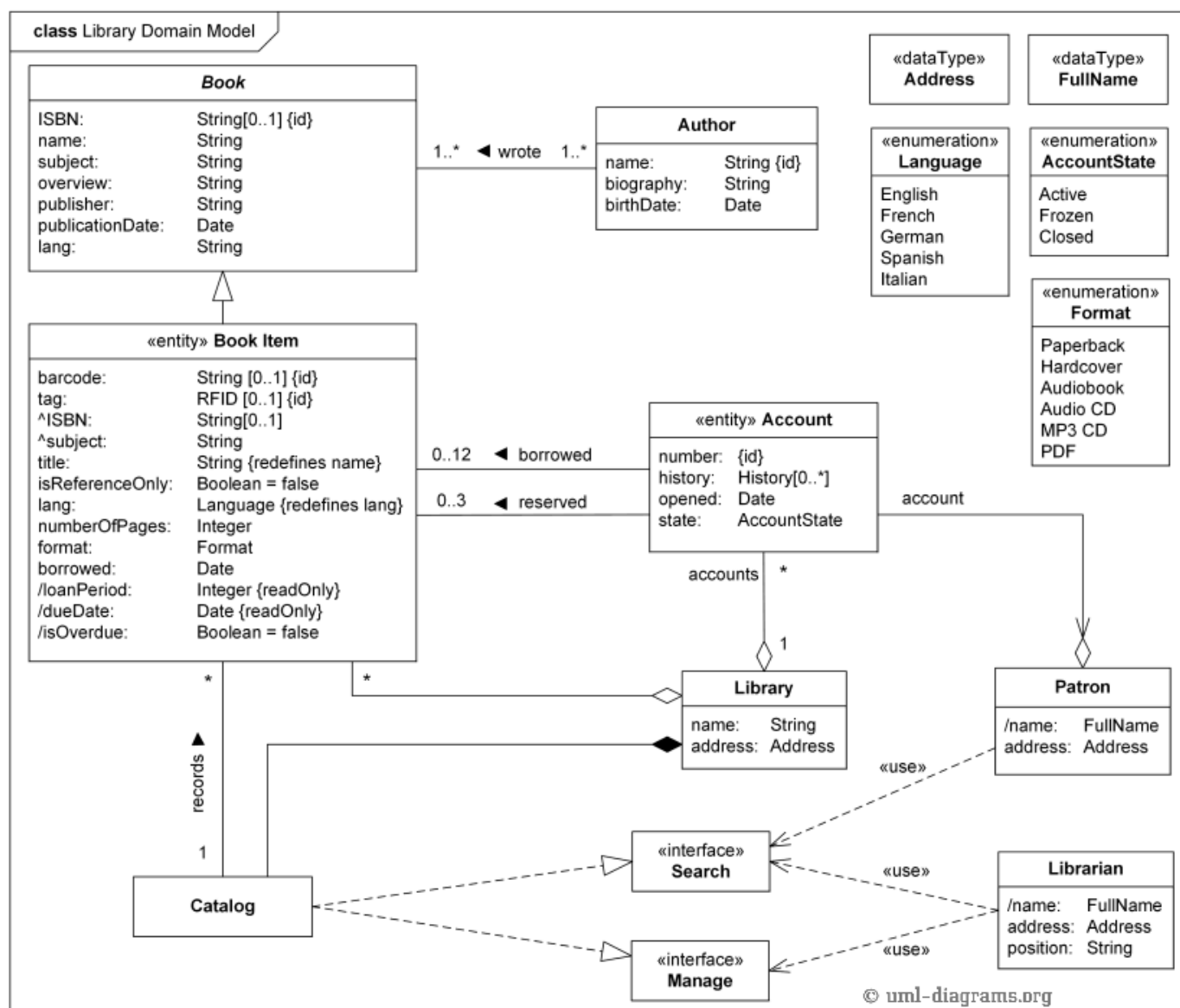
Los códigos de barras en los elementos de la biblioteca podrían ser reemplazados por etiquetas RFID. La etiqueta RFID puede contener el identificador del elemento, título, tipo de material, etc. Es leída por un lector RFID, sin la necesidad de abrir la cubierta de libro o la caja del CD/DVD para escanearla con un lector de código de barras.

Los atributos tema e ISBN del libro de la biblioteca, se heredan de Libro y se muestran anteponiendo el símbolo ^. El atributo título redefine explícitamente nombre. Mientras que el tipo de los atributos es el mismo, el nombre es diferente. El atributo idioma se redefine de forma explícita con tipo diferente; el tipo original era una cadena de texto libre, mientras que el atributo redefinido es de clase Lenguaje, más específico; se ha utilizado redefinición explícita en este caso porque los tipos de atributo String y Lenguaje no están relacionados: el Lenguaje es un tipo enumeración.

La biblioteca tiene algunas reglas sobre lo que podría ser prestado y lo que es sólo para referencia. Hay también reglas sobre la cantidad de libros que se podrían pedir prestados y cuántos podrían reservarse.

Los atributos del libro de la biblioteca periodoDePréstamo, fechaDeVencimiento, y estáAtrasado son atributos derivados. El tiempo que un libro de la biblioteca puede ser prestado (período de préstamo) depende de la política de la biblioteca y varía en función del tipo de libro y de quién lo está pidiendo prestado. Por ejemplo, en una biblioteca de universidad los estudiantes de grado podrían pedir en préstamo un libro durante 30 días, los estudiantes de máster durante un trimestre, y el personal de la facultad durante un año. En una biblioteca pública el período de préstamo normal para un libro podría ser de tres semanas, mientras que podría reducirse a 2 semanas para los libros nuevos. La fecha de vencimiento (de devolución) de los libros se calcula a partir de la fecha de préstamo y el período de préstamo. Si la fecha de vencimiento ya ha superado la fecha actual, el indicador booleano estáAtrasado, que es falso de forma predeterminada, se establecerá a verdadero.

El catálogo de la biblioteca proporciona acceso a los usuarios de la biblioteca y al personal a todas las fuentes de información sobre los artículos de la biblioteca, permitiendo la búsqueda por un determinado autor, sobre un tema en particular, o en un formato en concreto. Le informa al usuario de dónde se pueden encontrar los materiales que satisfacen sus necesidades específicas.



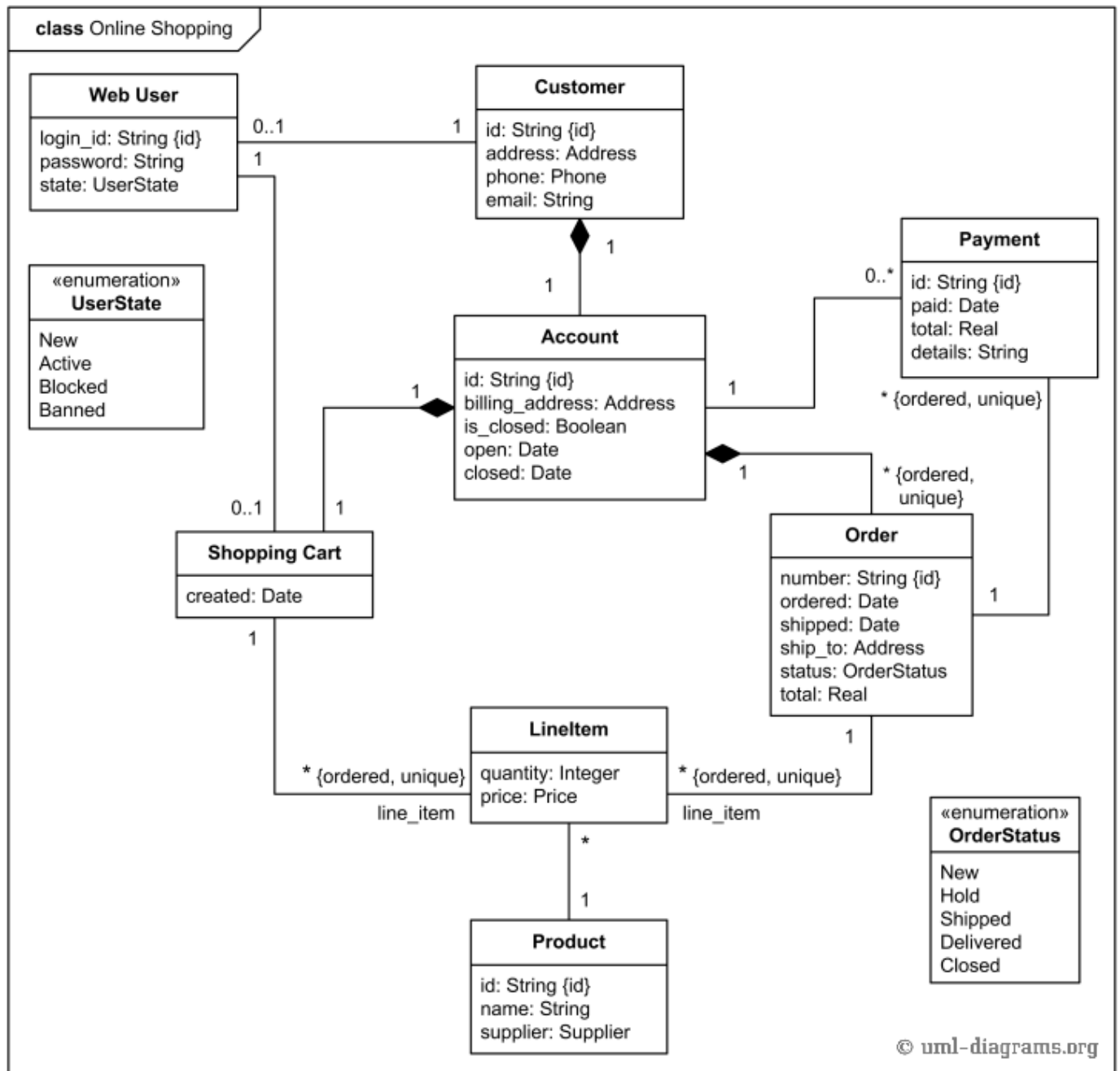
2.2 Modelo de dominio de un sistema de compras on-line

Este diagrama de clases UML muestra un modelo de dominio para un sistema de compras en línea. El propósito del diagrama es introducir algunos términos comunes, —un “diccionario” de compras on-line—, tales como Cliente, Usuario Web, Cuenta, Carrito de compra, Producto, Pedido, Pago, etc., y las relaciones entre ellos. Podría ser utilizado como una base común entre los analistas del negocio y los desarrolladores de software.

Cada cliente tiene un identificador único y está vinculado a una cuenta exactamente. La cuenta tiene un carrito de compra y unos pedidos. El cliente podría registrarse como usuario web para poder comprar artículos en línea. El cliente no está obligado a ser un usuario web porque las compras también se podrían realizar por teléfono o mediante pedidos por catálogo. El usuario web tiene un nombre de inicio de sesión que también sirve como identificador único. El usuario web podría estar en varios estados (nuevo, activo, bloqueado temporalmente o suspendido), y estar vinculado con un carrito de compra; el carrito de compra pertenece a una cuenta.

Una cuenta tiene los pedidos del cliente. El cliente puede no tener ningún pedido. Los pedidos de los clientes están ordenados y son únicos. Cada pedido puede tener varios pagos, o quizás a ninguno. Cada pago tiene un identificador único y se relaciona con exactamente una cuenta.

Cada pedido tiene un estado actual. Tanto el pedido como el carrito de compra tienen líneas asociadas a un producto específico. Cada línea se relaciona con exactamente un producto; a su vez, un producto podría estar asociado a muchas líneas o a ninguna.



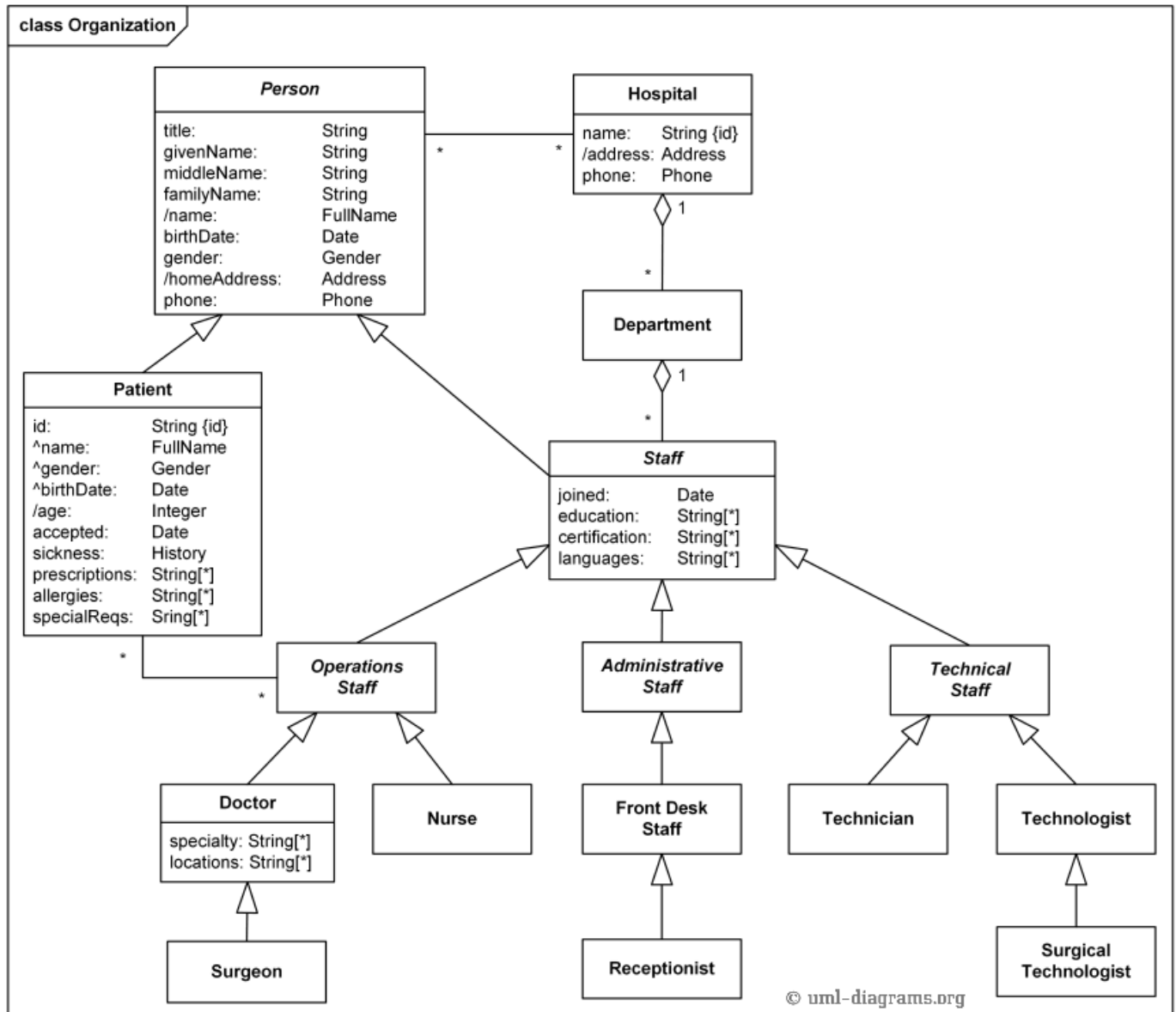
2.3 Modelo de dominio de la gestión de un hospital

El modelo de dominio para el sistema de gestión hospitalaria está representado por varios diagramas de clases. El propósito del diagrama es mostrar y explicar la estructura del hospital, el personal, las relaciones con los pacientes y la terminología de tratamiento del paciente.

En el siguiente diagrama una Persona podría estar asociada con diferentes hospitales, y un hospital podría emplear o servir a múltiples personas. La clase Persona tiene los atributos derivados nombre y domicilio. Nombre representa el nombre completo y podría estar compuesto del tratamiento, nombre y apellidos. La

clase Paciente tiene el atributo derivado edad que podría ser calculado a partir de su fecha de nacimiento y la fecha actual o la fecha de admisión en el hospital.

La clase Paciente hereda los atributos de la clase Persona. Varios atributos heredados, nombre, sexo y fecha de nacimiento, se muestran con el acento circunflejo antepuesto (^); es la nueva notación introducida en UML 2.5.

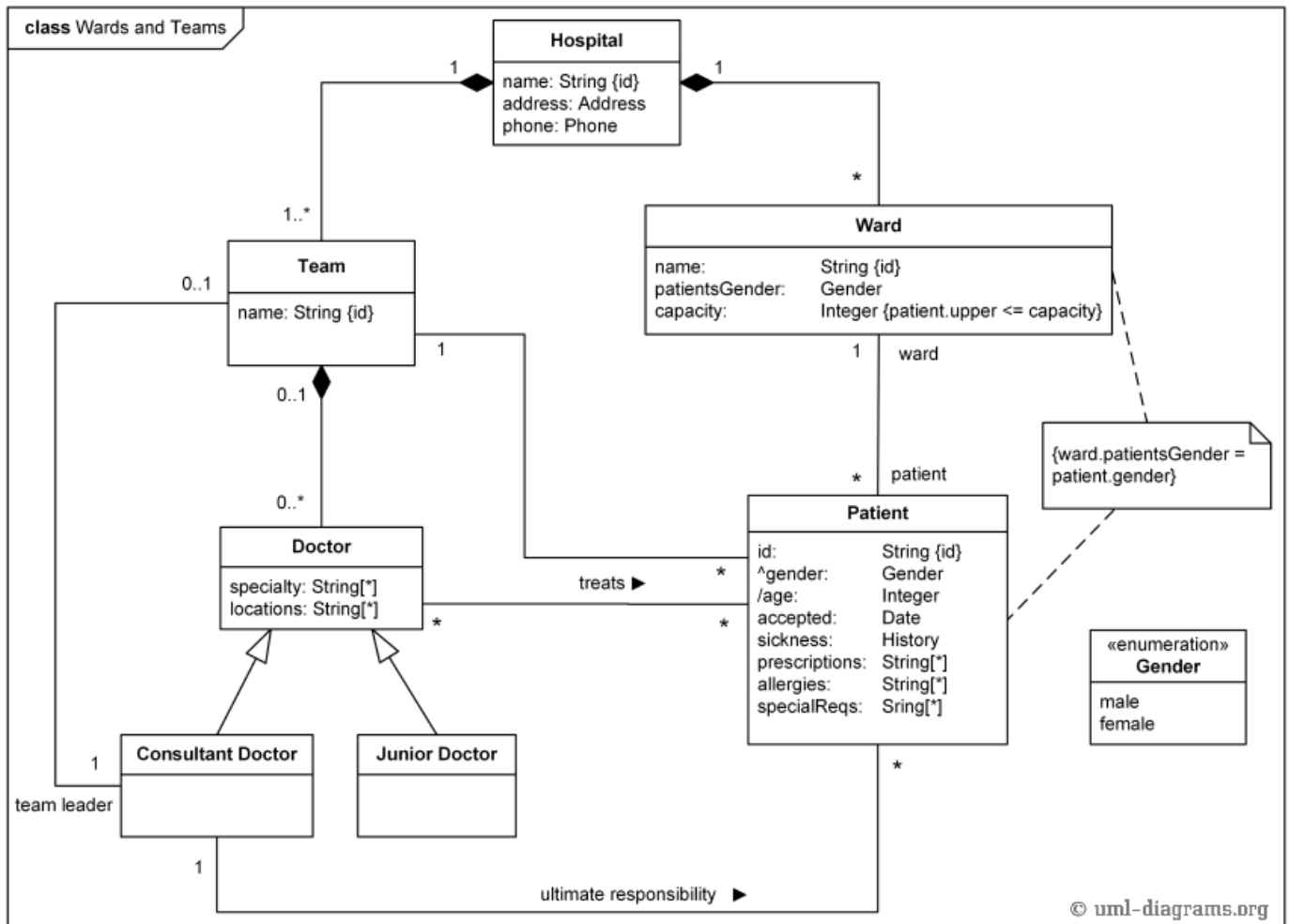


Unidad es una división de un hospital o un conjunto de habitaciones compartidas por los pacientes que necesitan el mismo tipo de atención. En un hospital, hay una serie de unidades, cada una de las cuales puede estar vacía o tener en ella uno o más pacientes. Cada unidad tiene un nombre único; el diagrama lo muestra con el uso del modificador {id} para el nombre de la unidad.

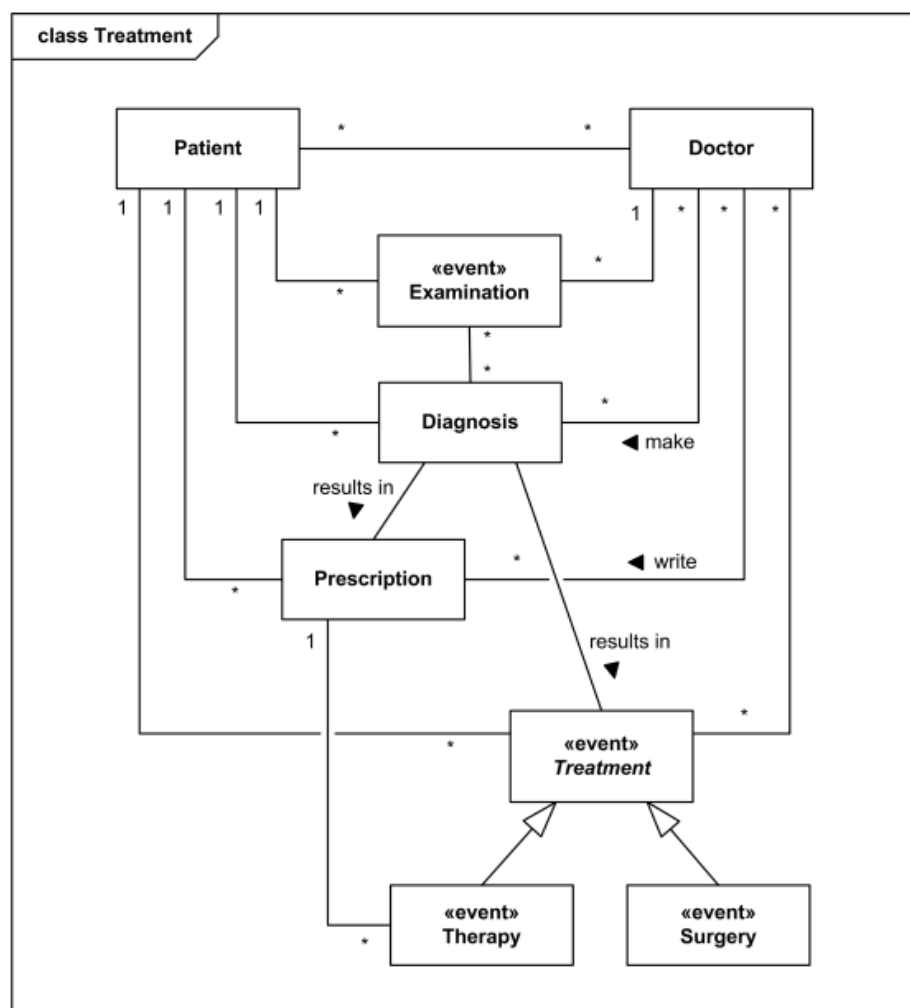
Las unidades se diferencian por el sexo de sus pacientes, es decir, unidades masculinas y femeninas. Una unidad sólo puede tener pacientes del sexo admitido en ella. El sexo se muestra como una enumeración. Así pues, unidad y paciente tienen una restricción en el sexo.

Cada unidad tiene una capacidad fija, que es el número máximo de pacientes que pueden estar en ella al mismo tiempo (es decir, la capacidad es el número de camas en la unidad). Diferentes unidades pueden tener diferentes capacidades.

Los médicos del hospital están organizados en equipos. Cada equipo tiene un nombre o código único (por ejemplo, Ortopedia o Pediatría) y está dirigida por un médico consultor (en el Reino Unido, Irlanda, etc.) o un médico de cabecera, también conocido como médico de planta (en los Estados Unidos). El médico consultor o el médico de cabecera es el médico de alto nivel que ha completado su formación especializada, residencia y prácticas de medicina en una clínica u hospital, en la especialidad aprendida durante la residencia; puede supervisar a becarios, residentes y estudiantes de medicina. El resto del equipo son todos los médicos en formación. Cada médico no puede ser miembro de más de un equipo.



Cada paciente está en una sola unidad y está bajo el cuidado de un único equipo de médicos. Un paciente puede ser tratado por cualquier número de médicos, pero todos ellos deben estar en el equipo que se ocupa del paciente. Un médico puede tratar cualquier número de pacientes. El líder del equipo acepta la responsabilidad última, legalmente, por la atención de todos los pacientes referidos a él, incluso sobre muchas de las decisiones que minuto a minuto están realizando sus subordinados.



2.4 Diagrama de clases que ilustra el uso de la API de cámara Android

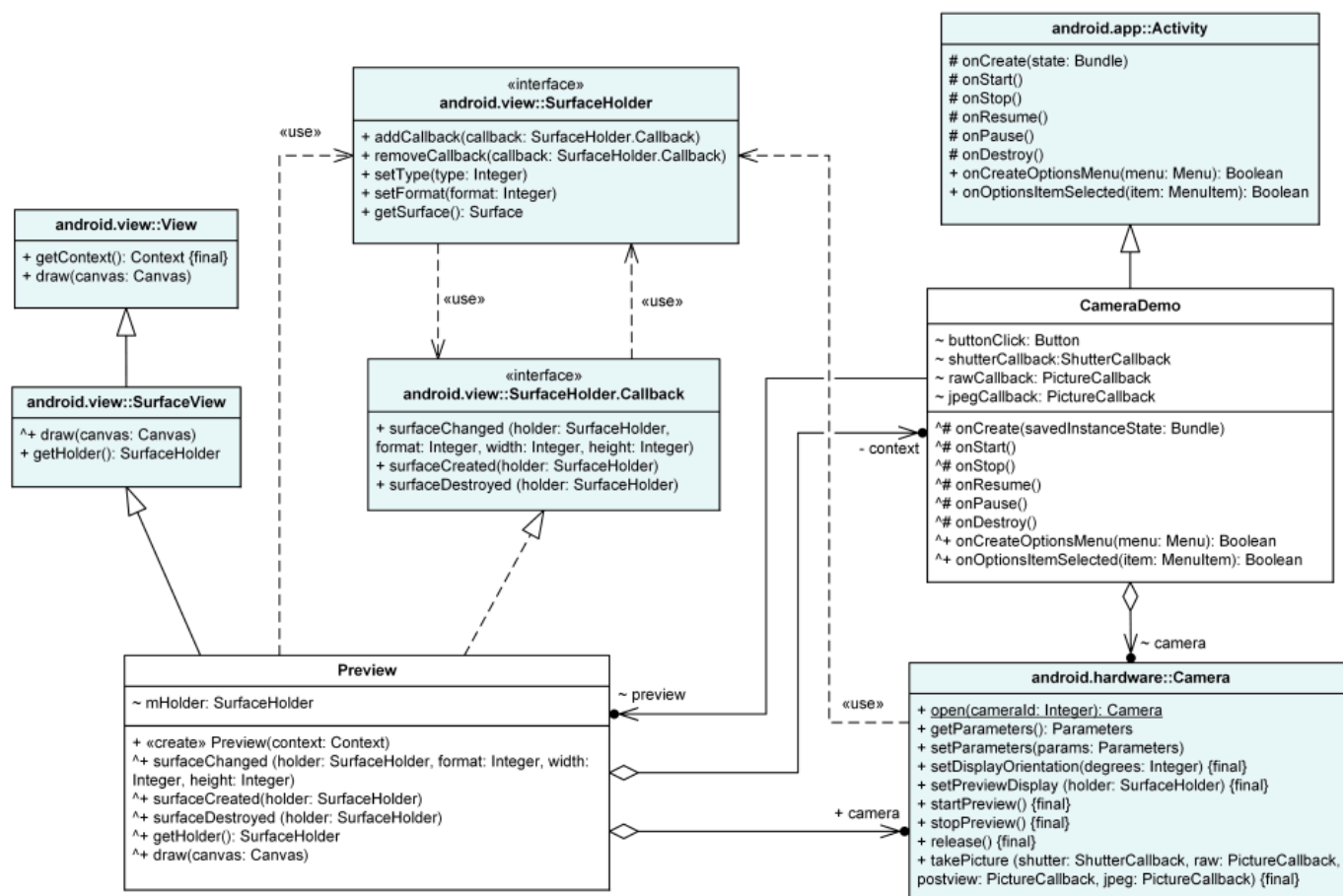
El siguiente es un diagrama de clases de nivel de implementación que muestra algunas clases que utilizan la API de la cámara de Android (Plataforma Android 3.1, Nivel de API 12). Este ejemplo de implementación se describe en “Using Android Camera API”.

La clase `CameraDemo` especializa `Activity` de Android. Una actividad es algo que el usuario puede hacer con Android; una actividad normalmente interactúa con el usuario, y la clase `Activity` se ocupa de la creación de la ventana en la que podemos colocar nuestra interfaz de usuario. La actividad `CameraDemo` creará un objeto `Preview` y mantendrá una referencia a él. `Preview` a su vez retiene una referencia a su actividad como su `Context`. El objeto `Preview` creará un objeto `Camera` y lo retornará a la actividad `CameraDemo`.

La clase `Camera` es una clase del paquete Java `android.hardware` (y ha sido declarada obsoleta en la API nivel 21, recomendándose en la nuevas aplicaciones utilizar la interfaz `android.hardware.camera2`). Se utiliza para obtener/crear objetos de la clase `Camera`, fijar los ajustes de captura de imagen, iniciar o detener la vista previa, tomar fotografías, etc. La clase `Camera` no es segura para programación multihilo, y debería ser utilizada desde un único hilo de eventos. Esta clase es un cliente del servicio `Camera`, que gestiona el hardware real de la cámara.

Habría que registrar algunos métodos de retro-llamada para que sean llamados por el objeto `Camera` asíncronamente después de que el método `takePicture` haya sido llamado: cuando el obturador se haya abierto, cuando haya sido tomada la fotografía, y cuando los datos de la imagen estén listos.

`Android.view.SurfaceHolder` es una interfaz implementada por los objetos que tienen una superficie de visualización. Permite controlar el tamaño y formato de la superficie, editar los píxeles de la superficie, controlar los cambios en la superficie, obtener acceso directo al objeto superficie, etc. La interfaz `SurfaceHolder.Callback` permite recibir información sobre los cambios en la superficie.



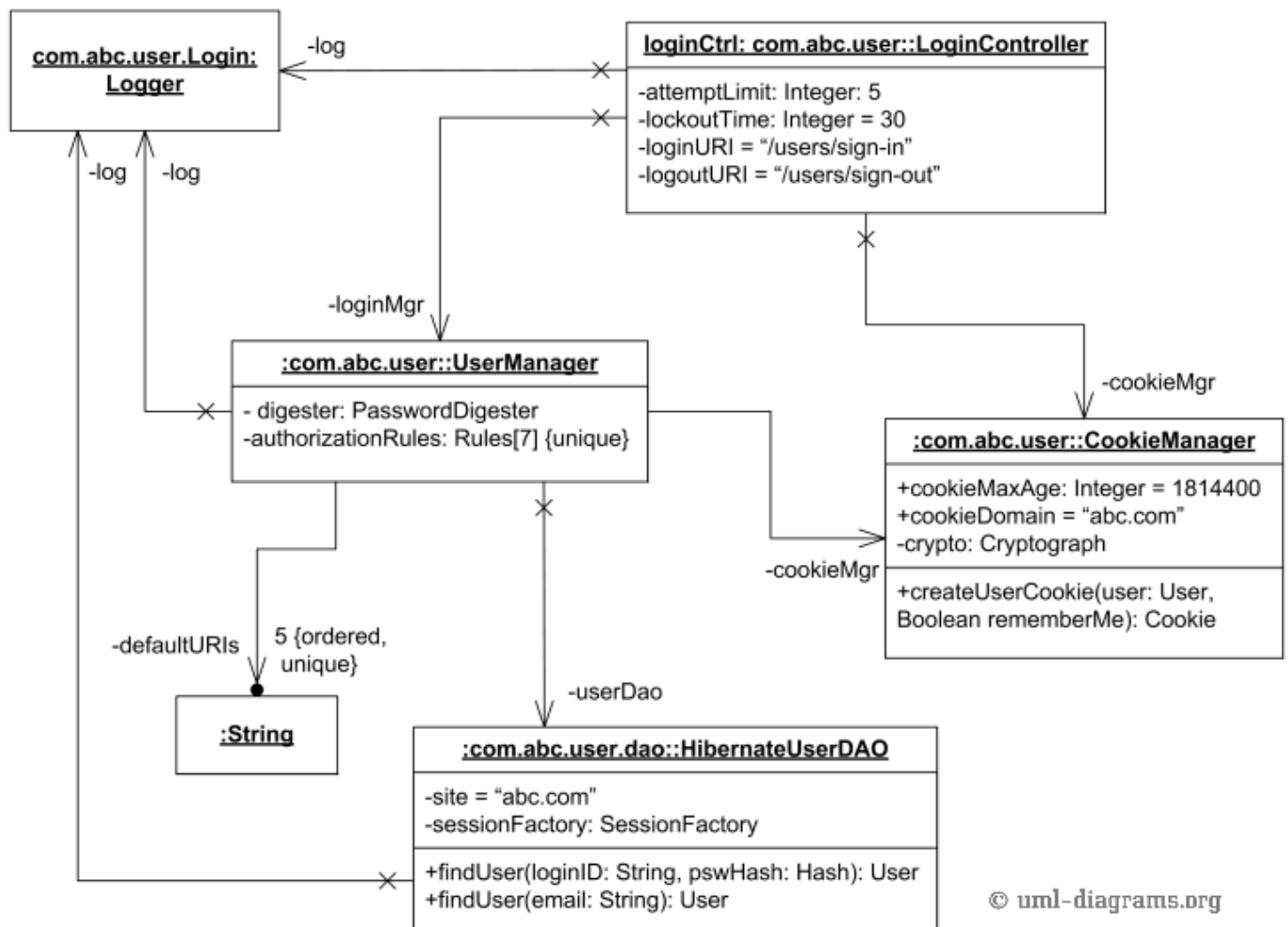
3 Diagrama de objetos

3.1 Controlador de inicio de sesión en un sistema de compras on-line

El siguiente diagrama de objetos muestra algunos objetos en tiempo de ejecución relacionadas con el proceso de inicio de sesión de un usuario web. La instancia `loginCtrl` de la clase `LoginController` tiene varios atributos de tipos `Integer` y `String` y las especificaciones de los valores correspondientes.

La instancia de `LoginController` también está asociada con instancias de `userManager`, `CookieManager` y `Logger`. `LoginController`, `userManager`, e `HibernateUserDAO` (*Data Access Object*) comparten una única instancia del `Logger`.

`userManager` tiene el atributo privado `defaultURIs` que es una colección ordenada (un array) de 5 cadenas únicas. La instancia de `CookieManager` tiene dos atributos públicos con los valores especificados. La mayoría de los enlaces son no navegables hacia atrás.



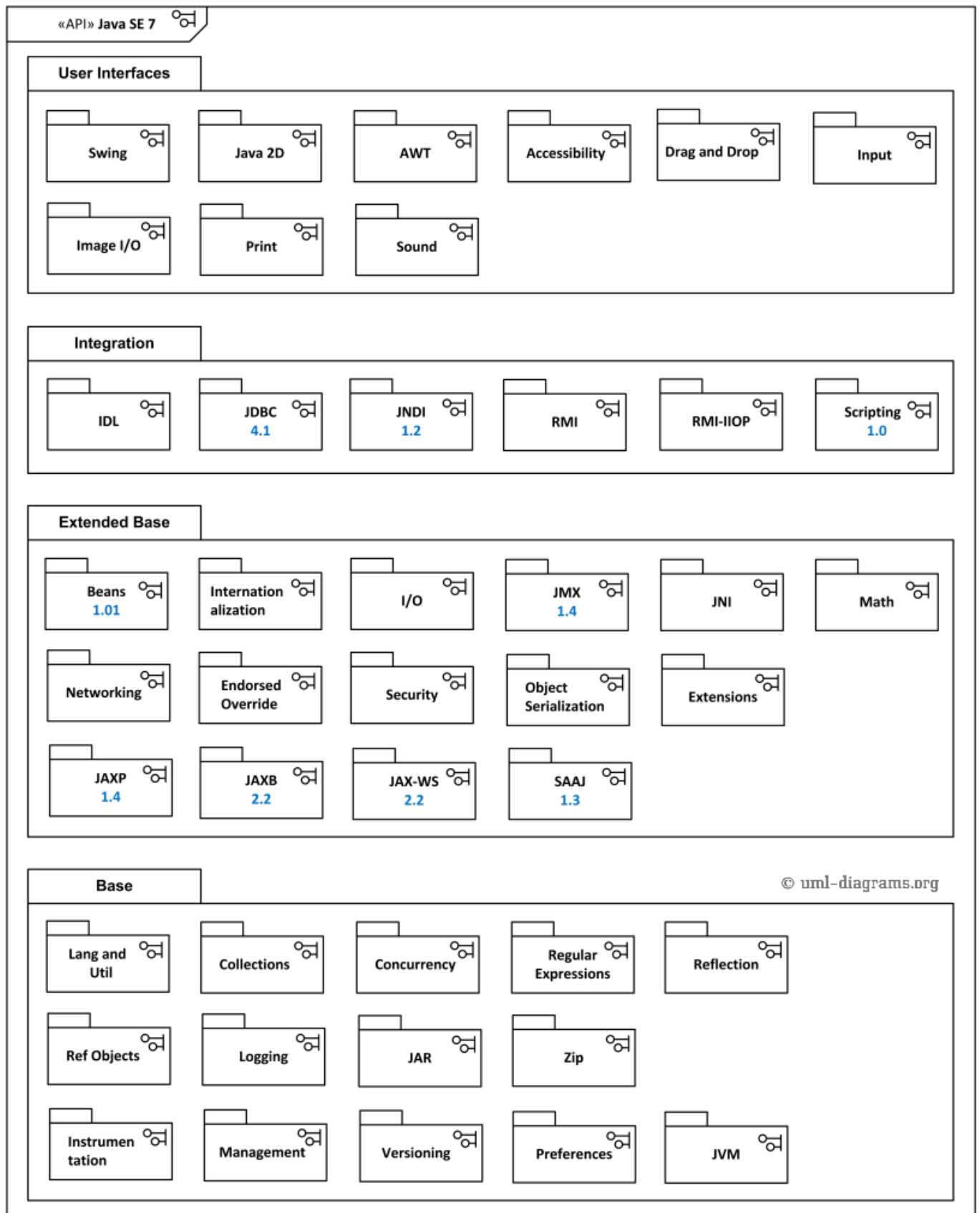
4 Diagrama de paquetes

4.1 API de la plataforma Java 7 Standard Edition

Este diagrama de paquetes UML representa la API de Java™ 7 Standard Edition (SE). La descripción oficial de la API es parte de la documentación de Oracle de la plataforma.

API (*Application Programming Interface*) es un término común en programación que por lo general se define como un conjunto de interfaces, clases y de algunas reglas que especifican cómo los clientes de la API podrían utilizar los servicios y/o recursos proporcionados por el componente software que implementa esa API.

La API Java SE 7 se compone de varios módulos. Nótese que UML 2.5 no proporciona la notación o un estereotipo para soportar el modelado de APIs. En el siguiente diagrama la API está anotada como un paquete estereotipado con el icono de doble interfaz, el cual no es un estereotipo de UML estándar.



4.2 API de Java Servlet 3.0

Este diagrama de paquetes UML 2.5 representa las más importantes interfaces y clases de la API Java TM Servlet 3.0. Esta API es una parte de la plataforma Java Enterprise Edition (“Java EE”), versión 6.

Téngase en cuenta que UML 2.5 no proporciona la notación o un estereotipo para soportar el modelado de APIs. En el siguiente diagrama la API está anotada como un paquete estereotipado como «API», el cual no es un estereotipo de UML estándar.

Un servlet es una clase Java que directa o indirectamente implementa la interfaz `Servlet` de la API Java Servlet. La documentación de Java EE por lo general se refiere a los servlets como componentes web. El ciclo de vida de los servlets es gestionado por un contenedor web. El contenedor web se proporciona como parte de un servidor web o de un servidor de aplicaciones Java EE.

La API Java Servlet 3.0 consta de cuatro paquetes: `javax.servlet`, `javax.servlet.http`, `javax.servlet.annotation` y `javax.servlet.descriptor`.

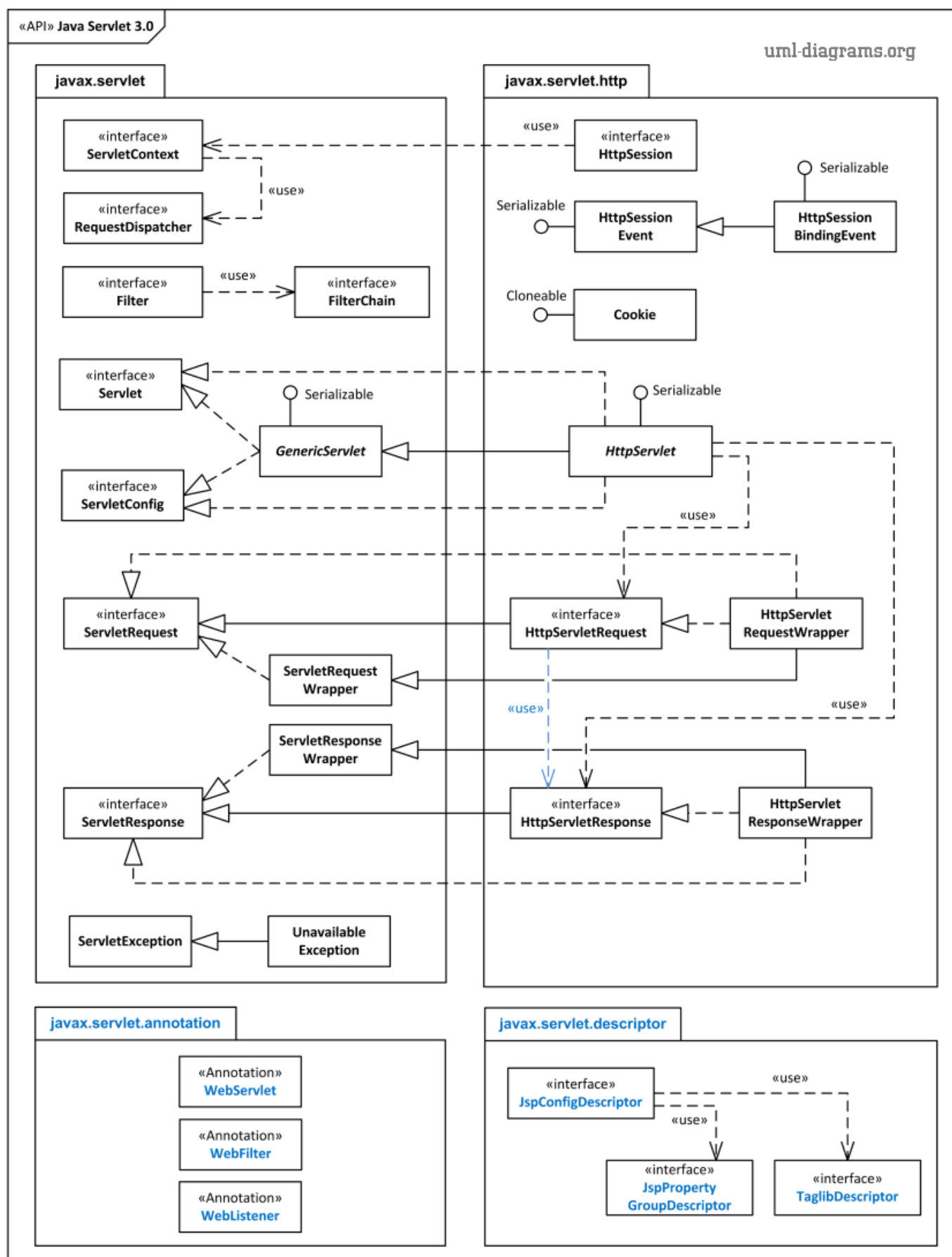
El paquete `javax.servlet` contiene una serie de interfaces y clases (tanto abstractas como concretas) que describen y definen los contratos entre una clase servlet y el entorno de ejecución proporcionado para una instancia de esa clase por un contenedor de servlets.

El paquete `javax.servlet.http` contiene las interfaces y clases especializadas para los servlets que soportan el protocolo HTTP y el entorno de ejecución correspondiente.

El paquete `javax.servlet.annotation` contiene una serie de anotaciones que permiten declarar servlets, filtros, oyentes y especificar los metadatos para el componente declarado. Nótese que una anotación en Java es un tipo especial de interfaz.

El paquete `javax.servlet.descriptor` proporciona interfaces que permiten el acceso mediante programación a la información de configuración de una aplicación web a partir de los descriptores `web.xml` y `web-fragment.xml`.

Dos clases abstractas en la API Java Servlet que implementan la interfaz `Servlet` son `GenericServlet` y `HttpServlet`. `HttpServlet` por lo general es implementada por los desarrolladores para programar los servlets específicos de la aplicación que soportan el protocolo HTTP.



4.3 Aplicación multi-capa

El siguiente diagrama de modelo UML representa un modelo de una aplicación en capas.

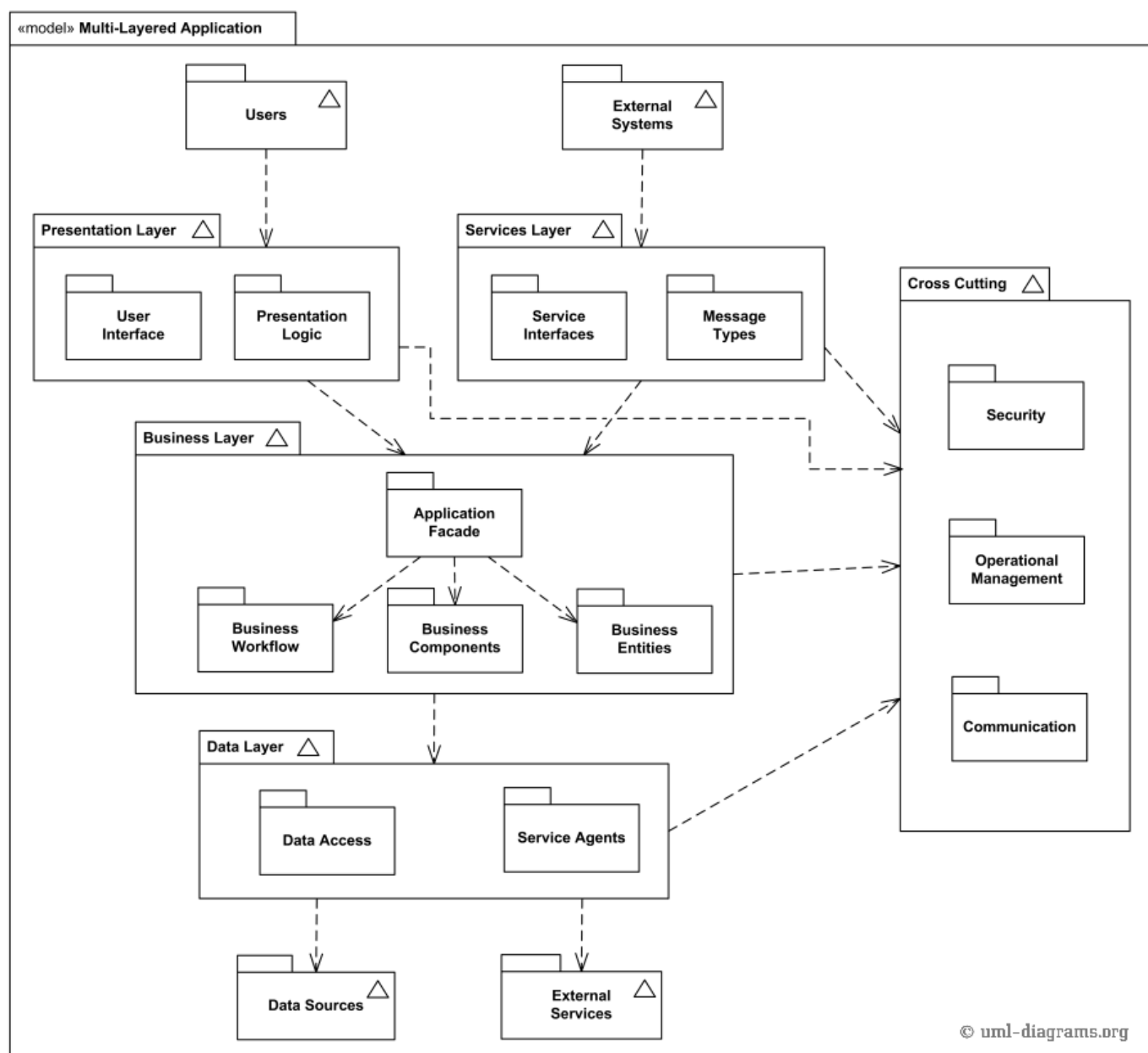
Un diagrama de modelo UML es una especialización de los diagramas de paquetes: es un diagrama de estructura auxiliar que muestra alguna abstracción o alguna vista específica de un sistema, para describir los aspectos arquitectónicos, lógicos o de comportamiento del sistema. El diagrama de modelo contiene un conjunto jerárquico de elementos que, juntos, describen el sistema; también puede contener un conjunto de elementos que representan el entorno del sistema, típicamente actores. Se podría mostrar, por ejemplo, la arquitectura de una aplicación multi-capa.

El diagrama de modelo mostrado está basado en la Guía de Arquitectura de Aplicaciones de Microsoft (2ª Ed.). De acuerdo con esta guía, las capas tienen que ver con la división lógica de los componentes y de la funcionalidad, y no tienen en cuenta la ubicación física de los componentes, mientras que los niveles describen la distribución física de la funcionalidad y de los componentes en servidores separados, computadoras, redes o ubicaciones remotas. Las capas pueden estar ubicadas en diferentes niveles, o pueden residir en el mismo nivel.

El modelo de aplicación muestra varias capas; presentación, servicios, negocios, datos y capas transversales. Todas las capas se representan como modelos UML.

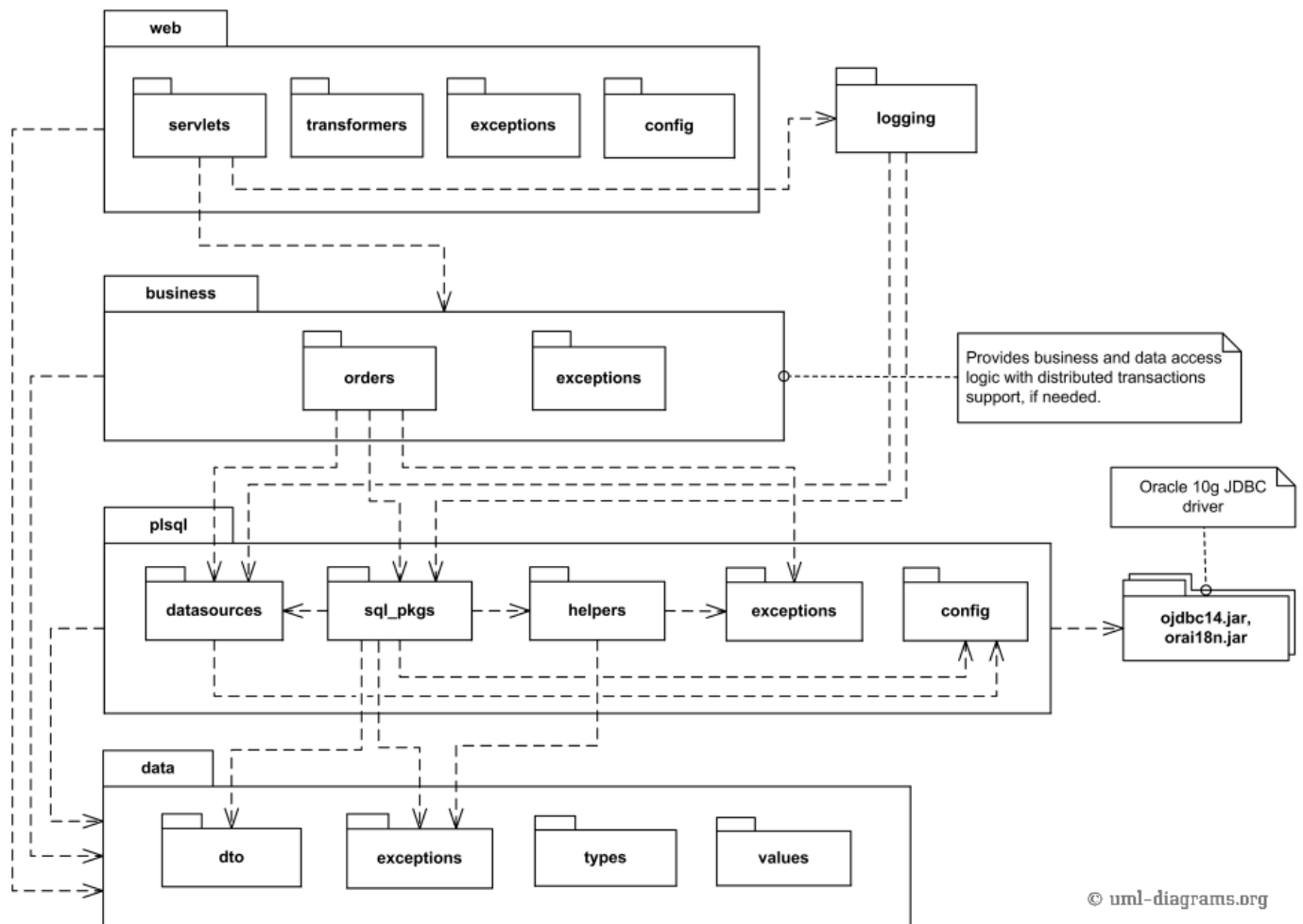
Los usuarios y los sistemas externos también están representados como modelos, y se comunican con capas de presentación y de servicios. El diagrama también muestra fuentes de datos, tales como bases de datos relacionales y agentes de servicios web que proporcionan acceso a datos y servicios externos o remotos que son consumidos por la aplicación.

La capa transversal contiene funcionalidad común que se extiende por capas y niveles. Esta funcionalidad típicamente soporta la autenticación, autorización, el almacenamiento en caché, la comunicación, la gestión de excepciones, el registro (*logging*) y la instrumentación, y la validación.



4.4 Arquitectura web multi-capa

Se muestra un ejemplo de diagrama de paquetes UML que representa una arquitectura web multi-capa. Las dependencias entre los paquetes se crean de tal manera que se eviten dependencias circulares entre los paquetes. Los paquetes de nivel superior dependen de paquetes de nivel inferior. Los paquetes que pertenecen a un mismo nivel podrían depender unos de otros. Objetos de transferencia de datos y excepciones comunes son utilizados por paquetes de los niveles superiores.



5 Diagramas de casos de uso

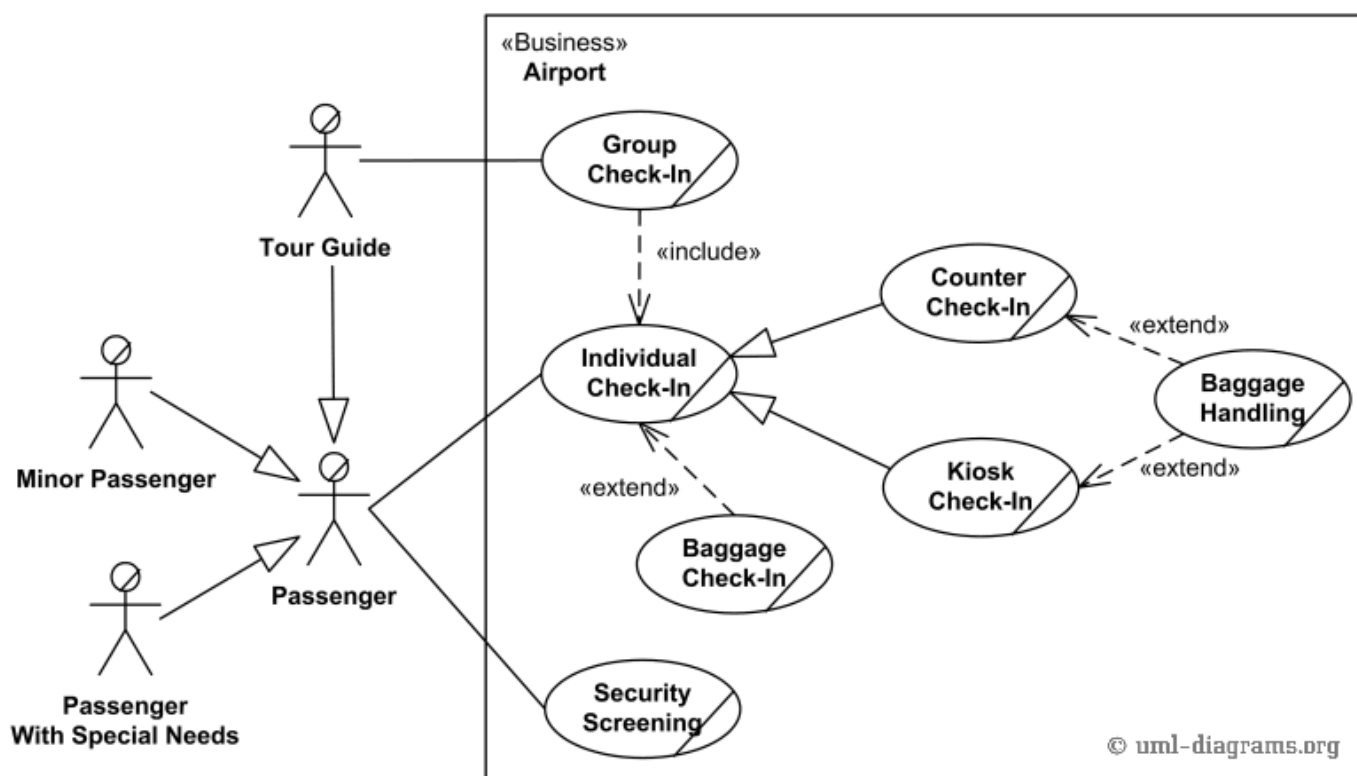
5.1 Área de embarque y control de seguridad

Este es un ejemplo de un diagrama de casos de uso del negocio, que por lo general se crea durante el modelado de negocio.

Los actores son Pasajero, Guía turístico, Menor, Pasajero con necesidades especiales (por ejemplo, discapacitado); todos juegan papeles externos en relación con el negocio de aeropuertos.

Los casos de uso son Facturación individual, Facturación en grupo (para grupos de turistas), Control de seguridad, etc.; representan las funciones o procesos de negocio que tienen lugar en el aeropuerto y que sirven a las necesidades de los pasajeros.

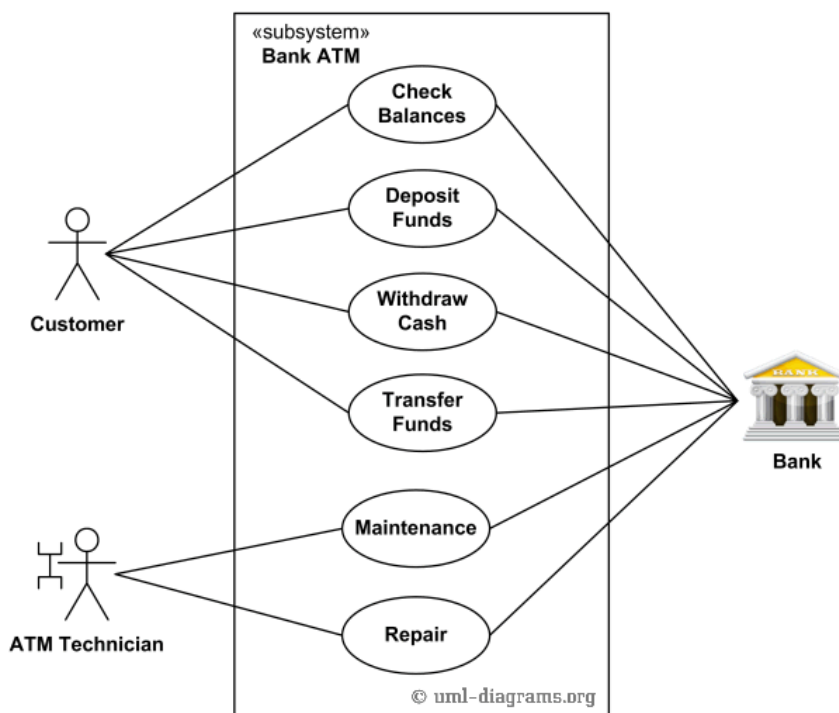
Los casos de uso Facturación de equipaje y Tratamiento de equipaje extienden el caso de uso Facturación individual, ya que los pasajeros podrían no tener equipaje, así la facturación de equipajes y su manejo son opcionales.



5.2 Cajero automático

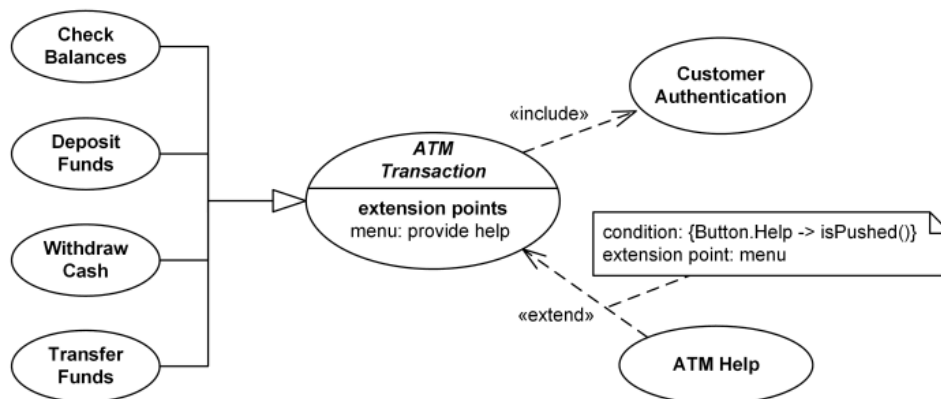
Un cajero automático (ATM) es un subsistema bancario que ofrece a los clientes del banco acceso a las transacciones financieras.

El cliente (actor) utiliza el cajero para Consultar los saldos de sus cuentas bancarias, Depositar fondos, Retirar efectivo y/o Transferir fondos (casos de uso). Los técnicos realizan el Mantenimiento y las Reparaciones. Todos estos casos de uso también implican el actor Banco.

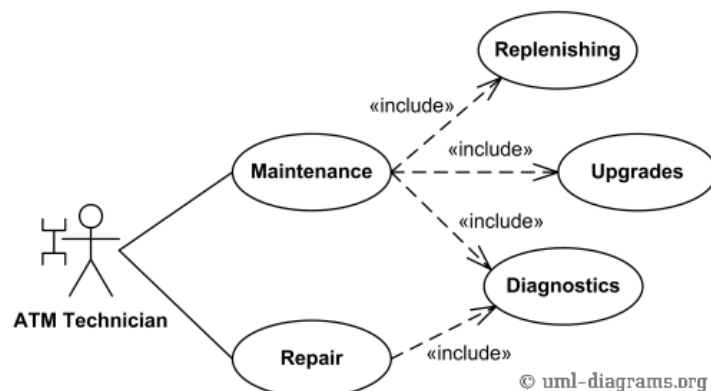


En la mayoría de los cajeros automáticos, el cliente se autentica mediante la inserción de una tarjeta e introduciendo un número de identificación personal (PIN). Se requiere el caso de uso Autenticación de cliente para cada transacción ATM, así que se muestra con una relación «include». La inclusión de este caso de uso, así como la generalización de las distintas transacciones, hacen de Transacción ATM un caso de uso abstracto.

El cliente puede necesitar alguna ayuda del cajero. Siempre que solicite ayuda, por ejemplo seleccionando la opción de menú Ayuda, el caso de uso Transacción ATM se extenderá en el punto de extensión Menú por el caso de uso Ayuda ATM

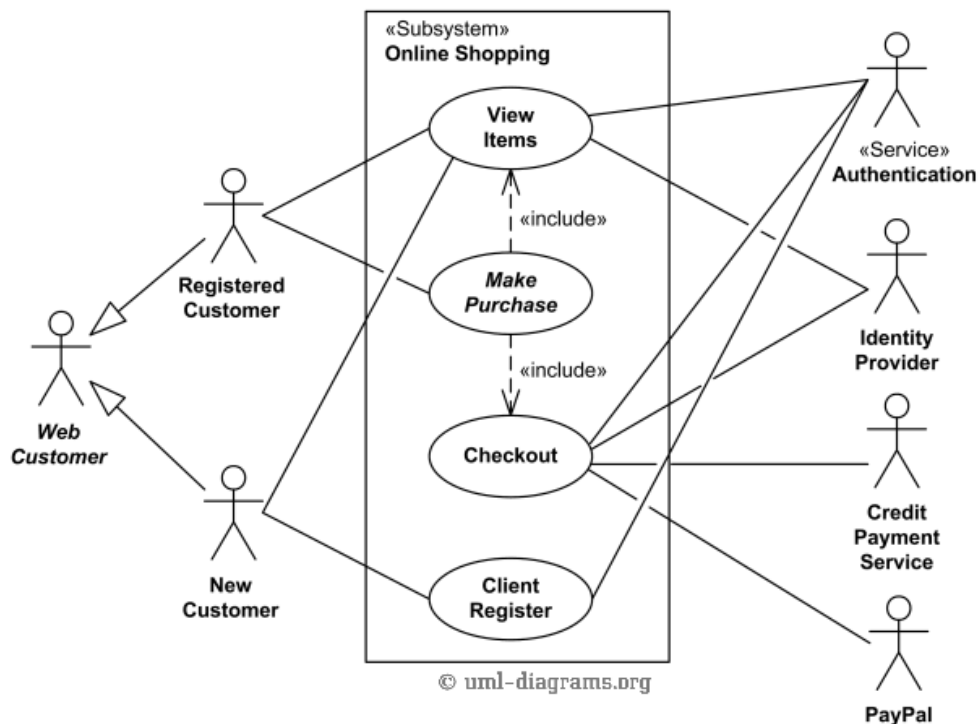


El técnico de ATM mantiene y repara los cajeros. El caso de uso Mantenimiento incluye Rellenar ATM con dinero en efectivo, tinta o papel de impresora, Actualizar el hardware, firmware o software, y Diagnosticar a distancia o in situ. El caso de uso Diagnosticar también está incluido en el caso de uso Reparar.



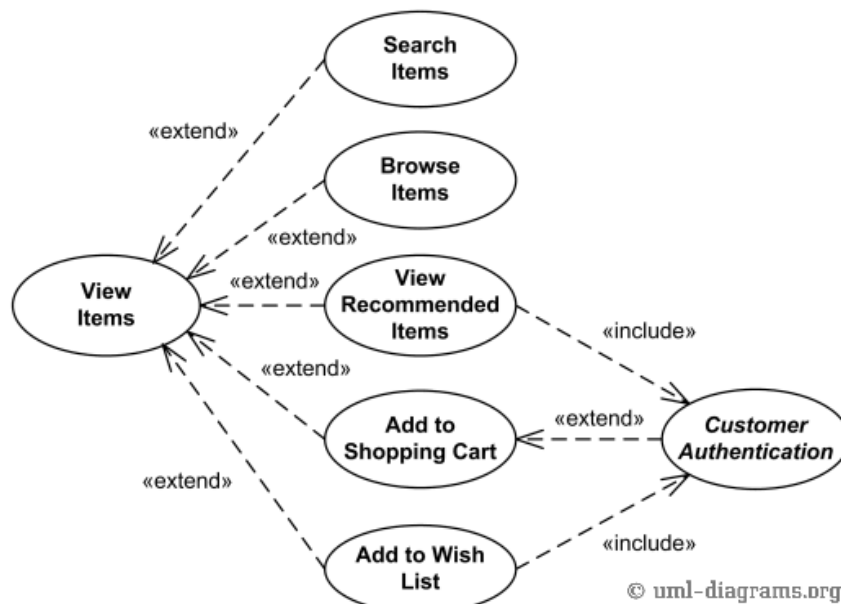
5.3 Compras on-line

El actor Cliente Web utiliza algún sitio web para realizar compras en línea. Los casos de uso de primer nivel son Ver artículos, Hacer compra y Registro de cliente. El caso de uso Ver artículos podría ser utilizado por el cliente como caso de uso de primer nivel si el cliente sólo quisiera encontrar y ver algunos productos. Este caso de uso también podría ser utilizado como parte del caso de uso Hacer compra. El caso de uso Registro de cliente le permite al cliente registrarse en el sitio web, por ejemplo para conseguir algunos cupones o ser invitado a ventas privadas. Téngase en cuenta que el caso de uso Pasar por caja es un caso de uso incluido no disponible por sí mismo, pues pasar por caja es parte de la compra.



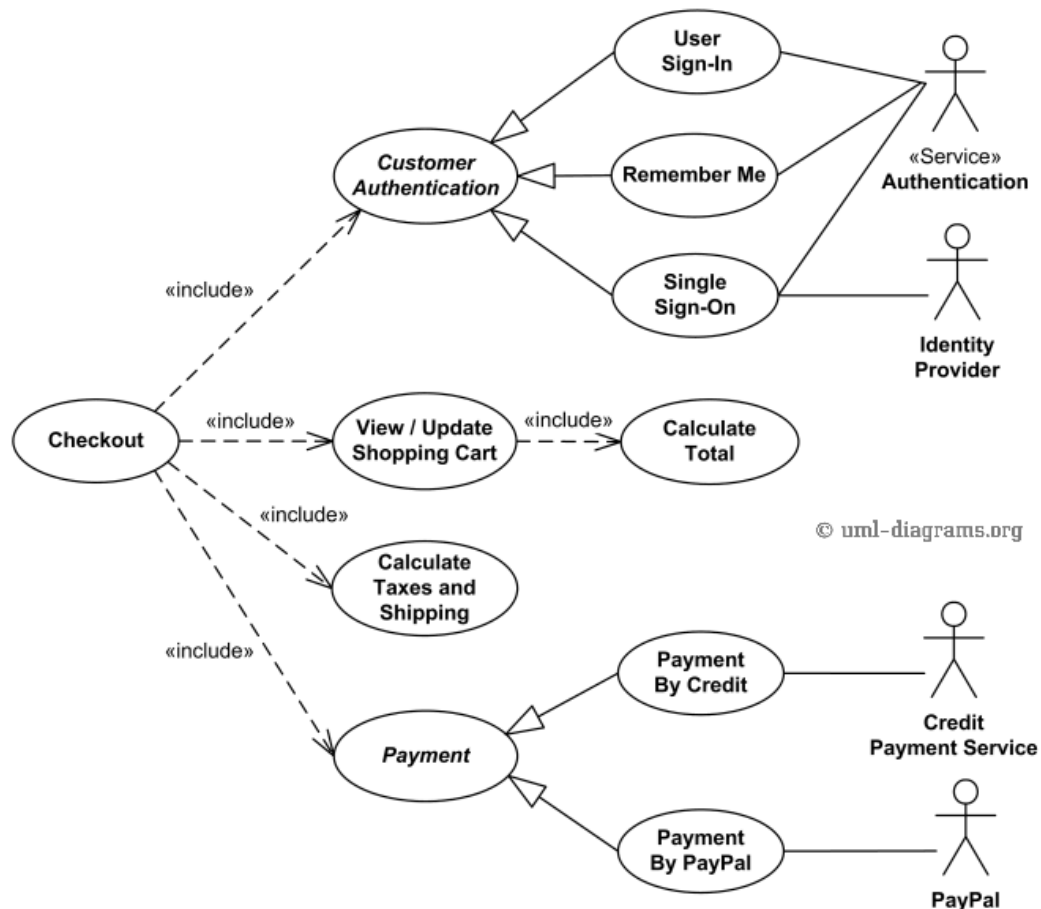
El caso de uso Ver artículos es extendido por varios casos de uso opcionales: el Cliente Web puede Buscar artículos, Navegar por el catálogo, Ver artículos recomendados para él, Añadir artículos al carro de compra y Añadir artículos a la lista de deseos. Todos estos casos de uso están extendiendo el caso de uso citado ya que proporcionan algunas funciones opcionales que permiten a los clientes encontrar un artículo.

El caso de uso Autenticación de cliente se incluye en Ver Artículos Recomendados y Añadir a lista de deseos, porque ambos requieren que los clientes sean autenticados. Al mismo tiempo, un artículo podría ser añadido al carro de compra sin necesidad de autenticación de usuario.



El caso de uso Pasar por caja incluye varios casos de uso requeridos. El Cliente Web debería estar autenticado. Esto podría hacerse a través de la página de Inicio de sesión de usuario, Cookie de autenticación de usuario (“Recuérdame”) o de Inicio de sesión único (Single Sign-On — SSO). El

Servicio de autenticación del sitio Web se utiliza en todos estos casos de uso, mientras que SSO también requiere la participación de un Proveedor de identidad externo.



El caso de uso Pasar por caja también incluye el caso de uso Pago que podría hacerse ya sea mediante el uso de tarjeta de crédito y un Servicio externo de pago a crédito o con PayPal.

6 Diagramas de actividad

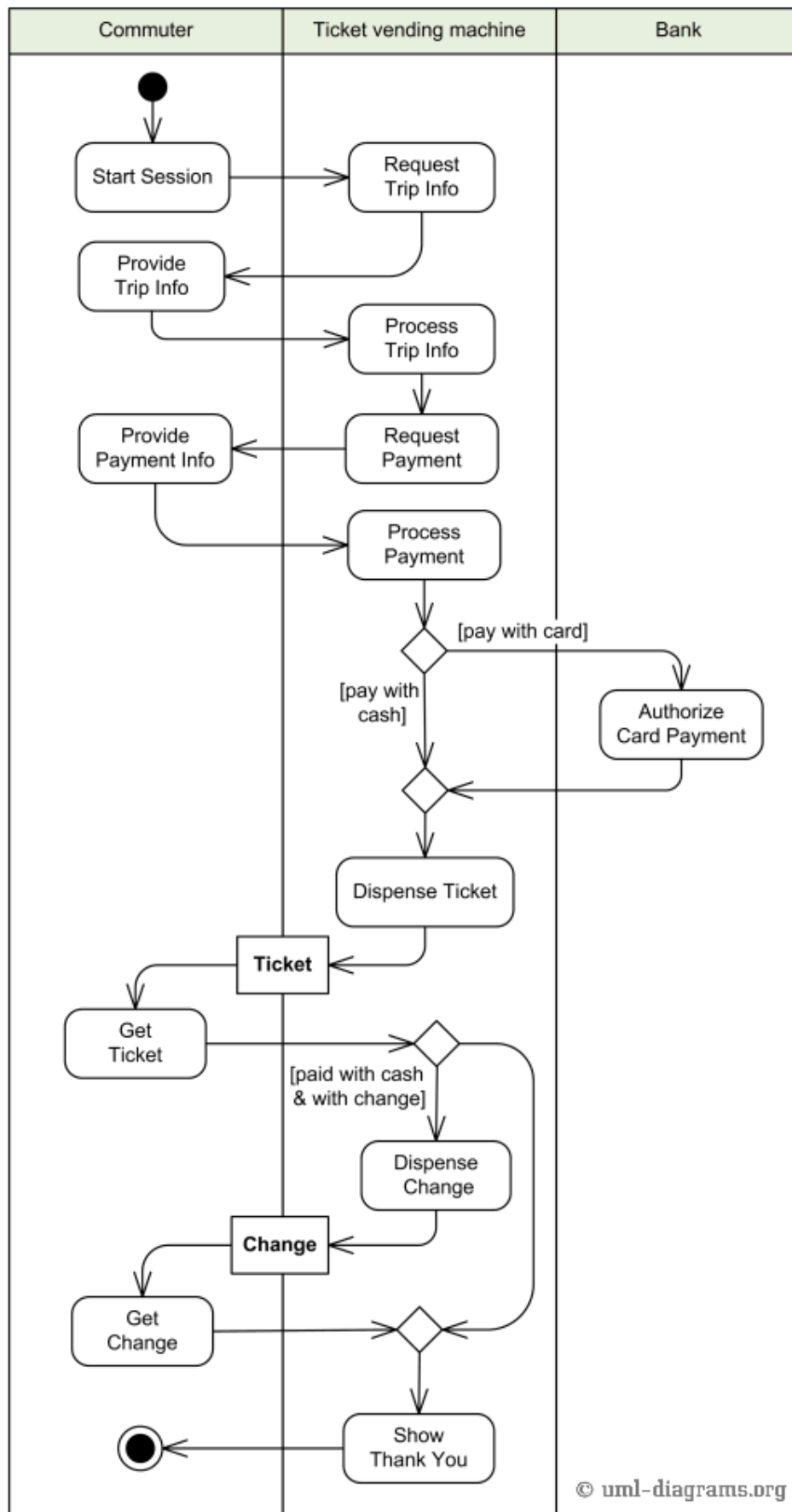
6.1 Máquina expendedora de billetes

Este es un diagrama de actividades UML que describe el comportamiento del caso de uso de compra de billetes.

La actividad se inicia por el actor Viajero que necesita comprar un billete. La máquina expendedora de billetes solicitará al Viajero la información del viaje. Esta información incluirá el número y tipo de billetes; por ejemplo, si se trata de un pase mensual, de un billete de ida y vuelta o sólo de ida, el destino, etc.

Basada en la información del viaje, la máquina expendedora calculará el pago debido y facilitará opciones de pago. Estas opciones incluyen el pago en efectivo o con tarjeta de crédito o débito. Si el viajero selecciona el pago con tarjeta, otro actor, el Banco, participará en la actividad para autorizar el pago.

Después de que el pago se ha completado, el billete se dispensará al Viajero. El pago en efectivo podría dar lugar a algún cambio, por lo que el cambio se le entregará al Viajero en este caso. La máquina expendedora mostrará alguna pantalla "Gracias" al final de la actividad.



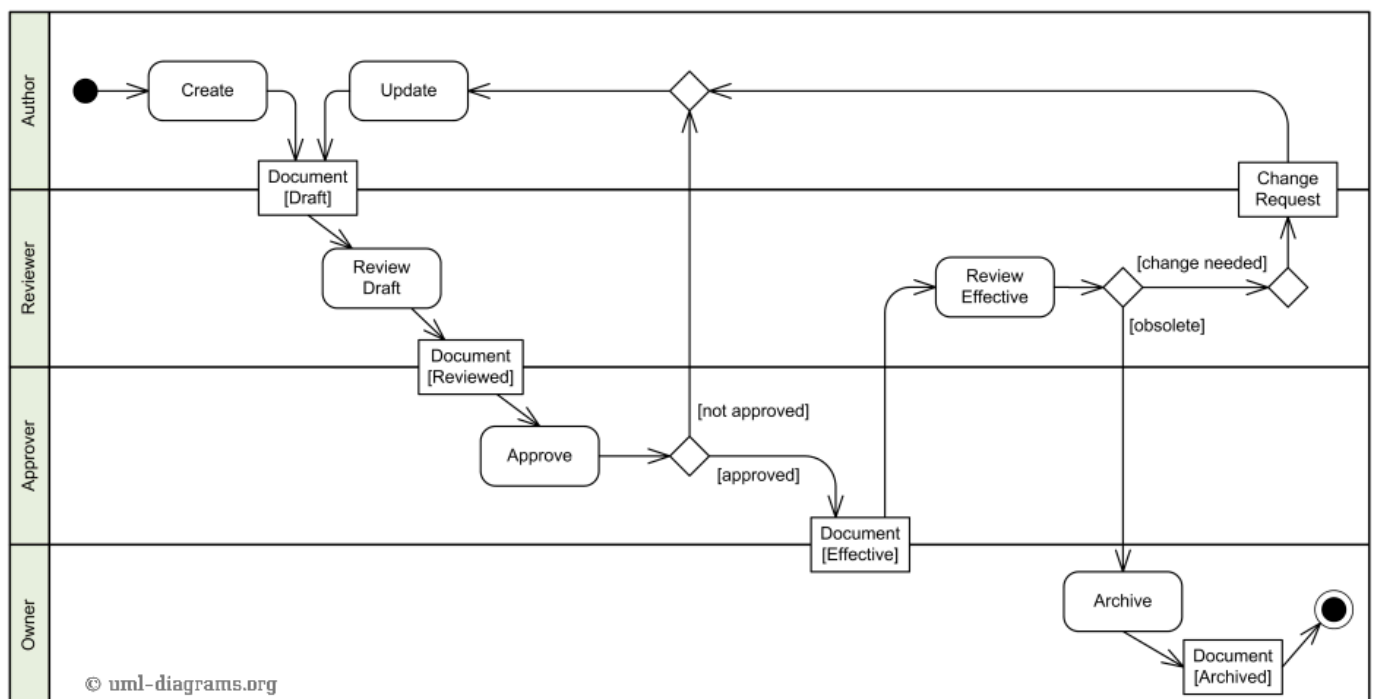
6.2 Proceso de gestión de documentos

Por lo general toda corporación importante requiere de algún tipo de proceso formal y debidamente comunicado de gestión de documentos.

Un documento pasa por diferentes estados: es creado, revisado, actualizado, aprobado, y en algún momento archivado. Diferentes roles que participan en este proceso son Autor, Revisor, Aprobador y Propietario.

Este diagrama de actividad muestra las responsabilidades de los diferentes roles y el flujo o secuencia de los cambios en el documento. Un tipo de diagrama alternativo —un diagrama de máquina de estados— también podría haberse utilizado en este caso para mostrar cómo un documento cambia su estado con el tiempo.

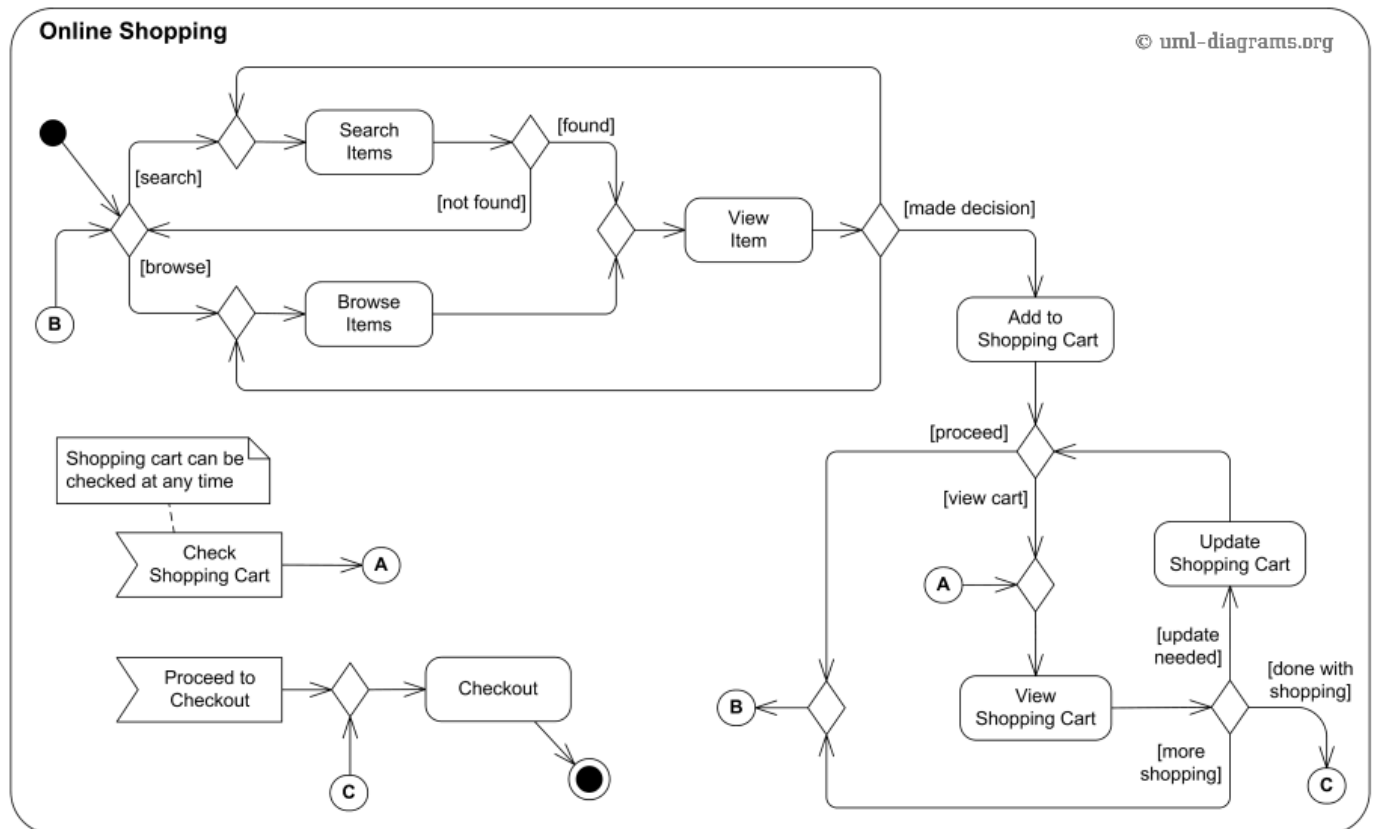
Téngase en cuenta que el objeto Documento no es el único nodo objeto mostrado en este diagrama de actividad. También hay otro objeto, la Solicitud de cambio, que se utiliza para comunicar los cambios al documento solicitados por el Revisor. Un diagrama de estados para el Documento sólo hubiera mostrado los estados del documento y sus transiciones, así que diagrama de actividad es útil cuando se ven envueltos diferentes roles y varios objetos.



6.3 Compra on-line

Un cliente on-line puede navegar o buscar artículos, ver un artículo específico, añadirlo al carro de compra, ver y actualizar el carro de compra, pasar por caja. El usuario puede ver el carro de compra en cualquier momento. El paso por caja se supone que incluye el registro de usuario e inicio de sesión.

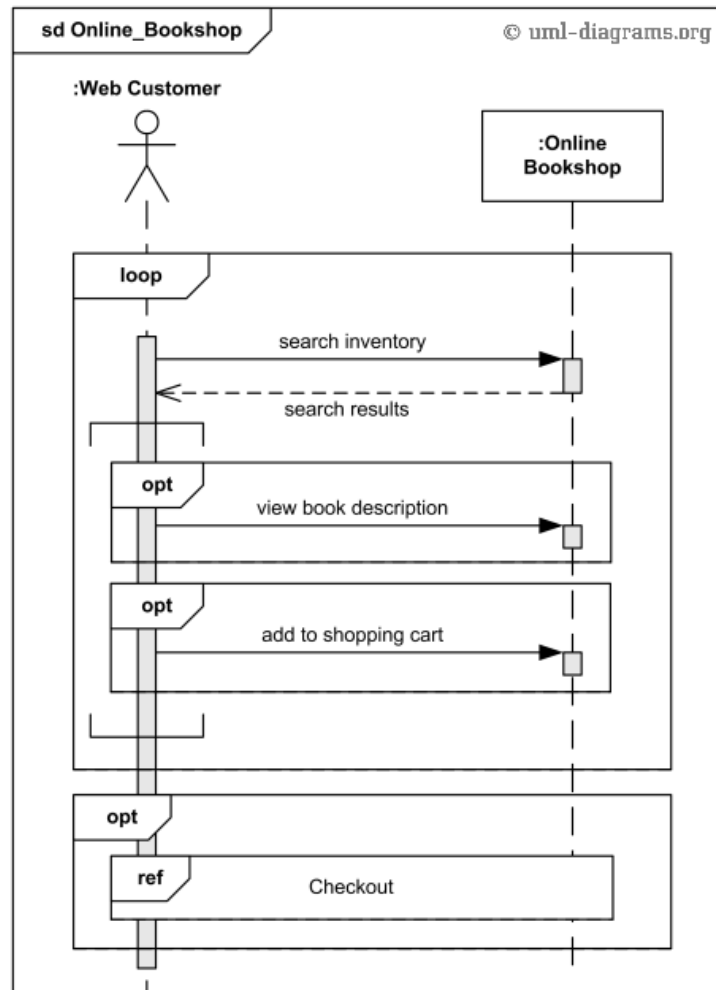
En este diagrama no se han utilizado particiones pues la mayor parte de las acciones las realiza el cliente.



7 Diagramas de secuencia

7.1 Compra on-line

Se muestra un diagrama de secuencia de alto nivel de una librería on-line. El cliente puede buscar en el catálogo de libros, ver la descripción de un libro seleccionado, añadir el libro al carro de compra y pasar por caja.



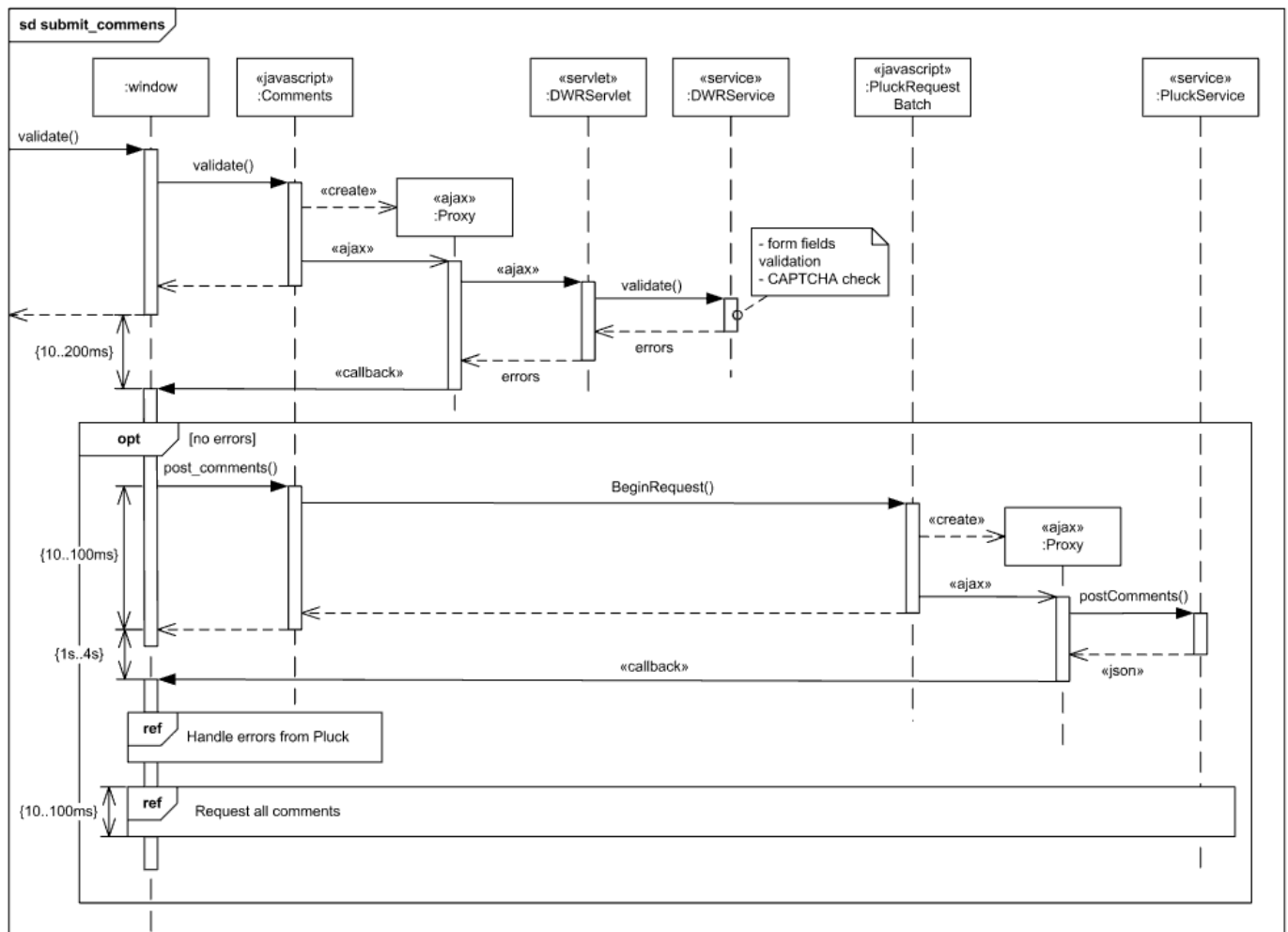
7.2 Proporcionar comentarios en Pluck

El diagrama de secuencia muestra cómo se introducen los comentarios del usuario sobre algún artículo utilizando diversas tecnologías AJAX.

Los comentarios presentados por el usuario son primero validados por el sitio web que aloja los artículos. La tecnología DWR (AJAX para Java) se utiliza para convertir los datos del formulario HTML (los comentarios de los usuarios) en objetos Java y en posibles errores de validación (que resultan en retro-llamadas JavaScript para comunicación de errores).

Los comentarios que parecen correctos son enviados al servidor Pluck, que aloja todos los comentarios a todos los artículos. La tecnología AJAX se utiliza también en este caso tanto para enviar el nuevo comentario como para recuperar la lista de todos los comentarios recientes (incluyendo el nuevo). Se utiliza JSON para obtener los comentarios.

Este diagrama también muestra algunas restricciones de tiempo (son ficticias y no describen ninguna comunicación real) Por ejemplo, de acuerdo con el diagrama, el tiempo de espera a la retro-llamada para un comentario enviado, varía de 1 a 4 segundos. Al mismo tiempo, solicitar todos los comentarios enviados debe tardar como máximo 100 mseg.



7.3 Autorización de usuario Web por Facebook

El siguiente diagrama de secuencia UML muestra cómo un usuario de Facebook (FB) puede ser autenticado en una aplicación web para permitir el acceso a recursos FB. FB utiliza el protocolo OAuth 2.0 que permite que una aplicación web (denominada “cliente”), que por lo general no es la propietaria del recurso FB pero que está actuando en nombre del usuario FB, solicite el acceso a los recursos controlados por ese usuario FB y alojados en el servidor FB. En lugar de utilizar las credenciales del usuario FB para acceder a recursos protegidos, la aplicación web obtiene un vale (*token*) de acceso.

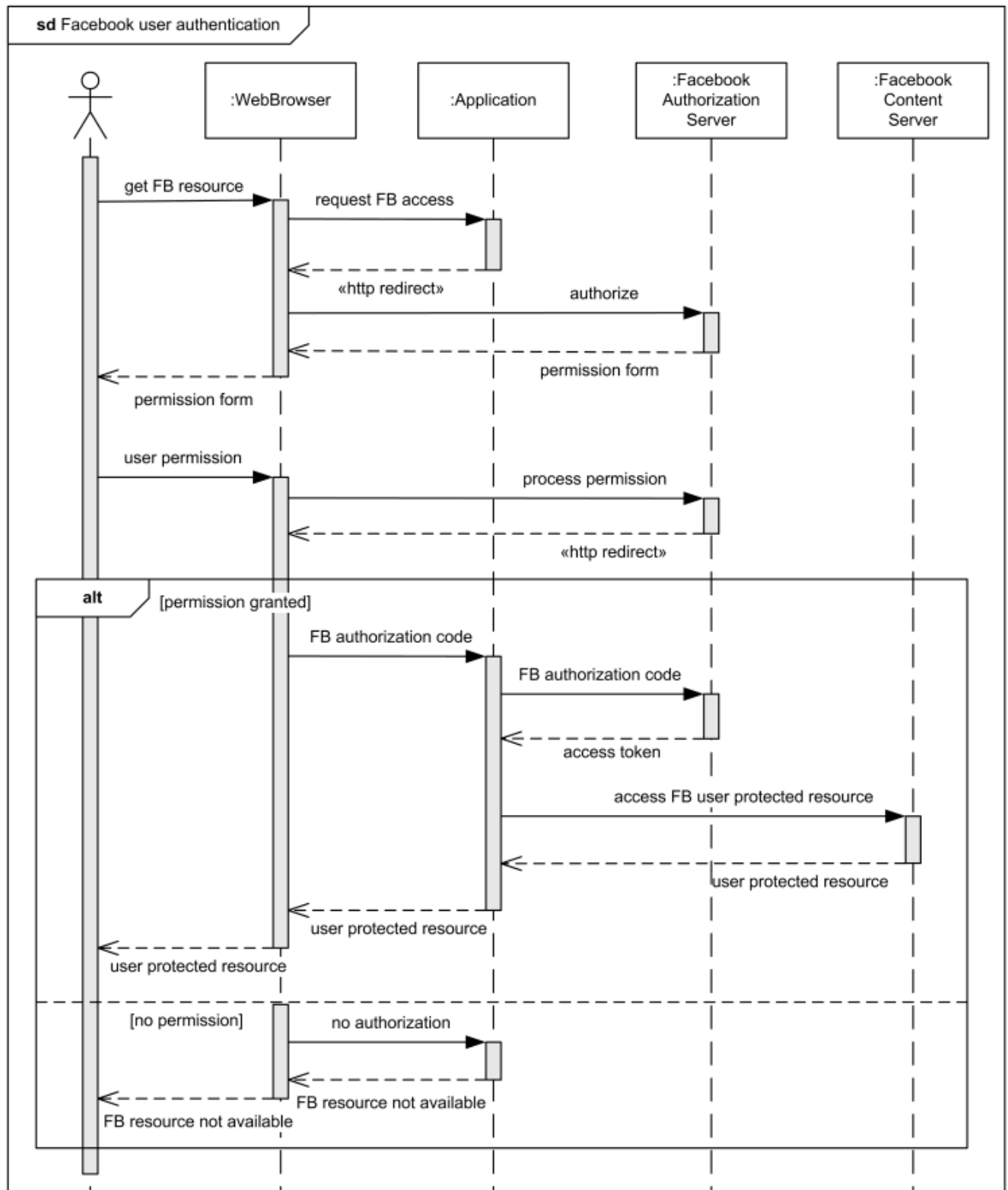
La aplicación web debe estar registrada en Facebook, de manera que tenga un ID de aplicación (`client_id`) y una clave secreta (`client_secret`). Cuando se recibe la solicitud de algunos recursos protegidos de FB, el navegador web (“agente de usuario”) es redirigido al servidor de autorización de Facebook con el ID de aplicación y la dirección URL a la que el usuario debería ser redirigido de nuevo después del proceso de autorización.

El usuario recibe el formulario de Solicitud de Permiso:

- Si el usuario autoriza a la aplicación para que acceda a sus datos, el servidor de autorización de Facebook redirige de nuevo al navegador a la URI que se especificó antes, junto con un código de autorización (“cadena de verificación”). El código de autorización puede ser intercambiado por la aplicación web para obtener un *token* de acceso OAuth.

Si la aplicación web obtiene el *token* de acceso para un usuario FB, puede realizar solicitudes autorizadas en nombre de ese usuario FB; las solicitudes a la API de Facebook Graph deberán incluir el *token* de acceso.

- Si el usuario no autorizó a la aplicación web, FB redirige de nuevo al navegador a la URI especificada antes, y añade el parámetro *error_reason* para notificar a la aplicación web que la solicitud de autorización fue denegada.



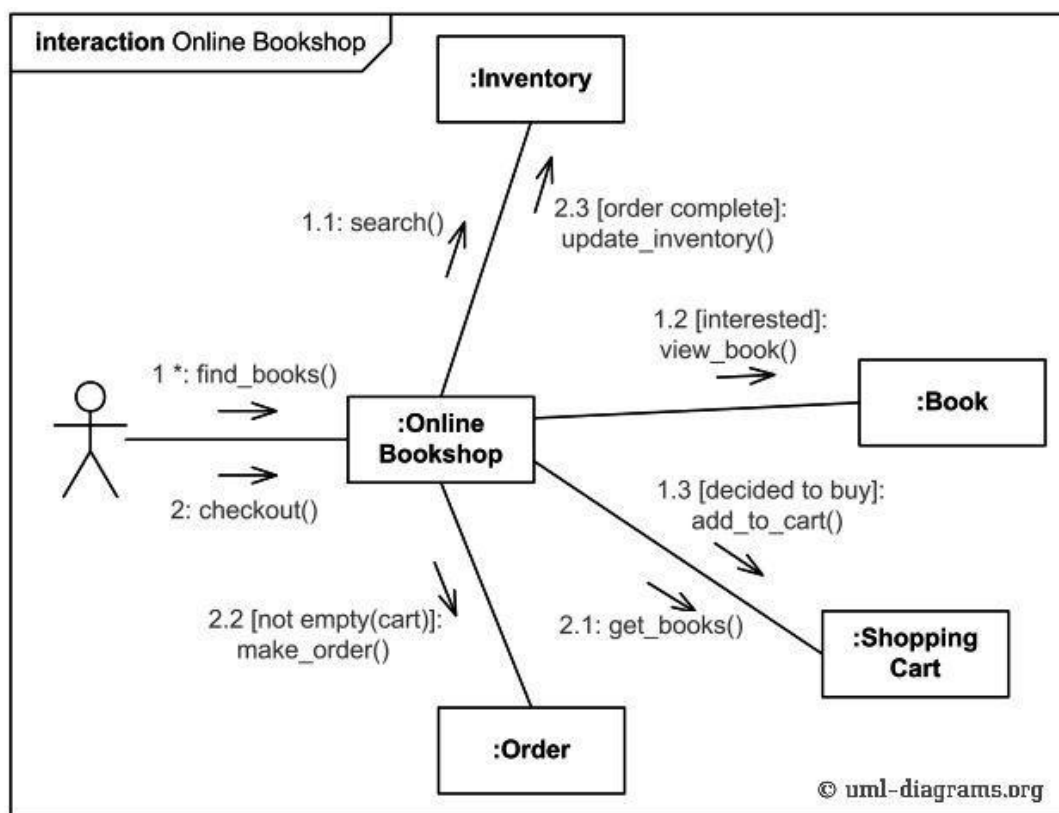
8 Diagramas de comunicación

8.1 Compra on-line

El diagrama de comunicación siguiente corresponde a una Librería on-line. El cliente Web (actor) puede buscar, ver y comprar libros:

1. La comunicación comienza con `1*:encontrar_libros()`, mensaje iterativo que podría repetirse un número no especificado de veces: el Cliente busca en el inventario de libros.
2. Si el Cliente se interesa por algún libro puede ver su descripción (`1.2[interesado]:ver_libro()`).
3. Si el Cliente decide comprar, debe agregar el libro a la cesta de la compra (`1.3[decidió comprar]:añadir_a_cesta()`).

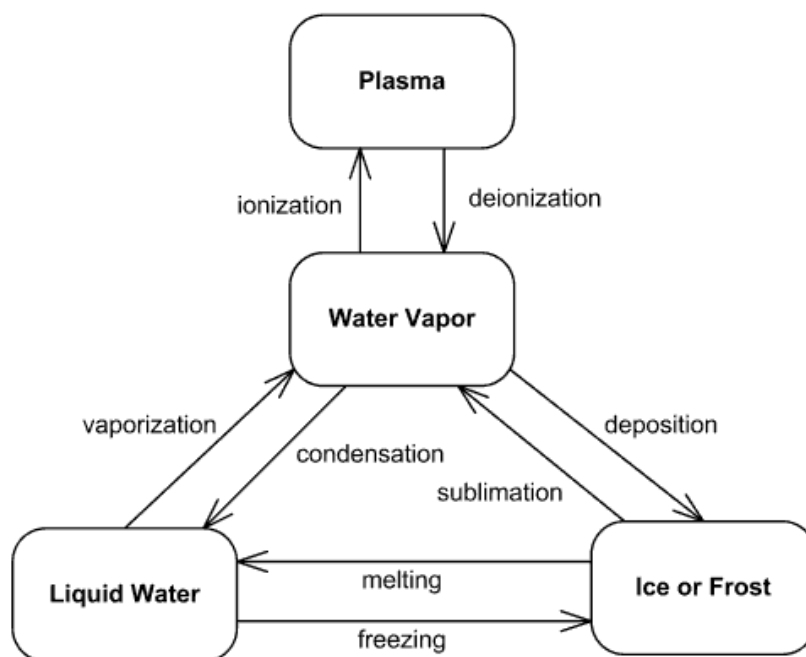
El paso por caja incluye obtener los libros de la cesta de la compra, crear el pedido, y actualizar el inventario si el pedido se completó.



9 Diagramas de estados

9.1 Estados del agua

El agua puede existir en varios estados - líquido, vapor, sólido, y plasma. Son posibles varias transiciones desde un estado a otro. Por ejemplo, la congelación es el cambio de fase del estado líquido al hielo. La condensación es el cambio de fase de estado de vapor a líquido. El vapor de agua puede convertirse directamente en hielo: es la deposición.



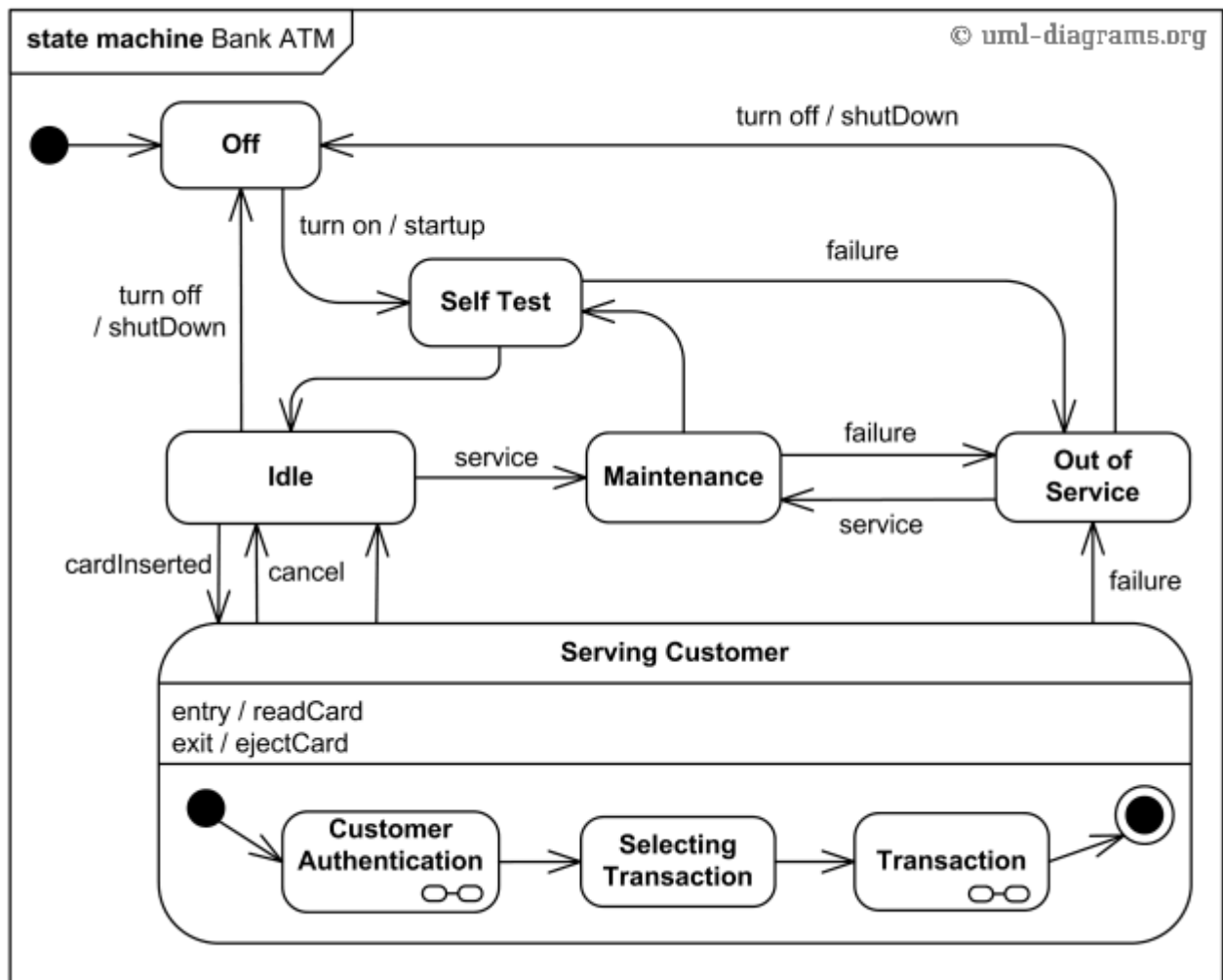
9.2 Cajero automático

El siguiente es un diagrama UML de máquina de estados de comportamiento que muestra la máquina de estados de nivel superior de un cajero automático (ATM).

Un ATM está inicialmente Apagado. Después de que la alimentación sea conectada, el ATM lleva a cabo la acción de puesta en marcha y entra en estado de Autocomprobación. Si la comprobación falla, el ATM entra en el estado Fuera de Servicio; si no, existe una transición sin disparador al estado de Reposo. En este estado el ATM espera la interacción del cliente.

El estado del ATM cambia desde Reposo a Atendiendo al cliente cuando el Cliente inserta una tarjeta bancaria en el lector de tarjetas del cajero automático. Al entrar en el estado Atendiendo al cliente, se lleva a cabo la acción de entrada leerTarjeta. Téngase en cuenta que la transición del estado Atendiendo al cliente al estado Reposo podría ser desencadenada por el evento Cancelar ya que el cliente podría cancelar la transacción en cualquier momento.

El estado Atendiendo al cliente es un estado compuesto por los sub-estados secuenciales Autenticación de cliente, Selección de transacción y Transacción. Autenticación de cliente y Transacción son a su vez estados compuestos lo cual se indica con el icono de descomposición oculta. El estado Atendiendo al cliente tiene una transición sin disparador al estado de Reposo después de que la transacción se termine. El estado también tiene la acción de salida expulsarTarjeta que libera la tarjeta del cliente al salir del estado, independientemente de lo que haya causado la salida del estado.



9.3 Cuenta de usuario en un sistema de compras on-line

Cada empresa que tiene clientes mantiene cuentas de clientes y define un ciclo de vida completo de cada cuenta desde su creación hasta su cierre. Existen diferencias en lo que son los estados en el ciclo de vida de una cuenta, y cuáles son las condiciones o hechos que provocan que la cuenta cambie su estado.

Aquí se proporciona el ciclo de vida de una cuenta de usuario en el contexto de las compras on-line; se muestra como un diagrama UML de máquina de estados de protocolos.

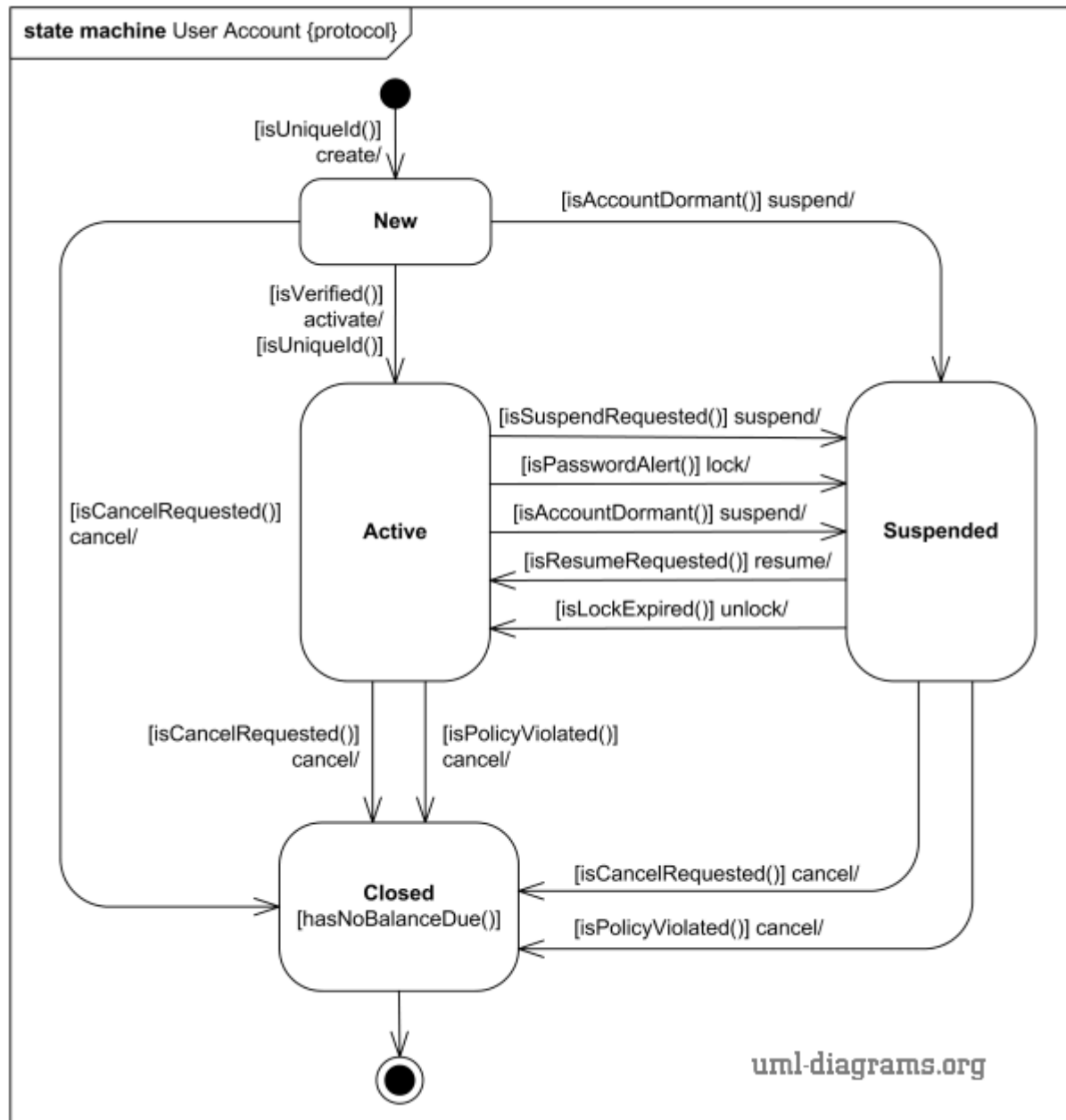
Para que se cree una cuenta de usuario, tiene que cumplir con algunos requisitos iniciales. Por ejemplo, el identificador de usuario (usado como nombre de inicio de sesión) debe ser único, al menos respecto de las cuentas existentes. Después de que la cuenta se cree, puede que tenga que ser verificada. La verificación depende de la empresa y podría incluir la comprobación del correo electrónico, el teléfono y/o la dirección. Si la cuenta no se verificó durante un período de tiempo predefinido, la cuenta podría ser considerada una cuenta suspendida.

Las cuentas nuevas, suspendidas o activas podrían ser canceladas en cualquier momento a petición del cliente. Téngase en cuenta que la condición previa para esto incluye por lo general el pago de los saldos pendientes, y podría requerir algún estado de cuenta separado (o sub-estado) para manejar este caso.

La cuenta de usuario podría ser suspendida por razones de seguridad, de forma manual o automática. Por ejemplo, el sistema de detección de intrusiones del sitio web bloqueará la cuenta de usuario durante un período de tiempo predefinido si hubo varios intentos fallidos de iniciar sesión utilizando una contraseña

de cuenta incorrecta. Después de que el tiempo de bloqueo de la cuenta expire, se activará la cuenta de nuevo automáticamente.

Algunas cuentas de usuario podrían estar inactivas durante un largo periodo de tiempo. La política de la empresa o las reglas de negocio podrían requerir mover al estado de suspensión esas cuentas que han estado durmientes durante un año o dos.



10 Bibliografía

- <http://www.uml-diagrams.org>